Chapter 1: Introduction to Parallel Processing

1.1 Parallel Computer Structures & Uniprocessor Parallelism

- Is there any form of parallelism in a uniproprocessor system? If yes, explain the different forms of parallelism used in a uniprocessor system and if no, explain why it is not feasible. *(2025 Fall)*

Yes. *Although a uniprocessor has only one CPU and is classified as* ***SISD****, it can still use internal parallelism to improve performance. The main forms are* ***Instruction-Level Parallelism (ILP), Data-Level Parallelism (DLP), and Thread-Level Parallelism (TLP).***

1. Instruction-Level Parallelism (ILP)

- ***Pipelining:*** *Divides instruction execution into stages such as Fetch, Decode, Execute, and Write-back.* ***It overlaps multiple instructions to increase throughput.***
- ***Superscalar Execution:*** *Uses multiple functional units to execute more than one independent instruction per cycle.* ***It increases processing speed.***
- ***Out-of-Order Execution:*** *Executes independent instructions before earlier stalled instructions.* ***It keeps the processor's functional units busy.***
- ***Branch Prediction & Speculative Execution:*** *Predicts the next branch and executes instructions before the result is known.* ***It reduces pipeline stalls.***
- ***Register Renaming:*** *Uses different physical registers to remove false dependencies.* ***It allows more instructions to execute in parallel.***

2. Data-Level Parallelism (DLP)

- ***SIMD/Vector Processing:*** *Performs the same operation on multiple data elements at once.* ***It speeds up data-intensive operations.***
- ***Parallel Arithmetic Hardware:*** *Uses fast arithmetic circuits such as carry-lookahead adders and fast multipliers.* ***It reduces computation time.***
- ***Memory Interleaving:*** *Divides memory into multiple modules for overlapping memory accesses.* ***It increases memory bandwidth.***

3. Thread-Level Parallelism (TLP)

- ***Hardware Multithreading/SMT:*** *Allows instructions from different threads to use processor resources concurrently.* ***It hides latency and improves processor utilization.***

Conclusion

Thus, a uniprocessor can exploit parallelism through ***ILP, DLP, and TLP****, even though it has only one CPU. These techniques improve performance by allowing independent instructions, data operations, or threads to* ***overlap or execute concurrently****.*

- What are the different mechanisms of parallelism used in a uniprocessor system? Explain. *(2024 Fall)*
- What is parallelism in a computer system? Explain different techniques used to enhance the performance in a uniprocessor system? Why is the importance of parallelism increasing day by day? Explain with suitable examples. *(2025 Fall Mock)*

1.2 Architectural Classification & Applications

- What is parallel processing? Why is the demand of parallel processing increasing day by day? Explain with examples. *(2025 Fall)*
- Flynn's classification *(Short Note, 2025 Fall)*
- Explain the architectural classification of computer systems based on Flynn's taxonomy. Provide examples for each category and discuss their relevance in parallel computing. *(CA / Old)*

***Flynn's Taxonomy**, proposed by **Michael J. Flynn in 1966**, classifies computer architectures based on the number of **instruction streams** and **data streams** they process.*

- ***Instruction stream:** Sequence of instructions executed by the processor.*
- ***Data stream:** Data on which the instructions operate.*

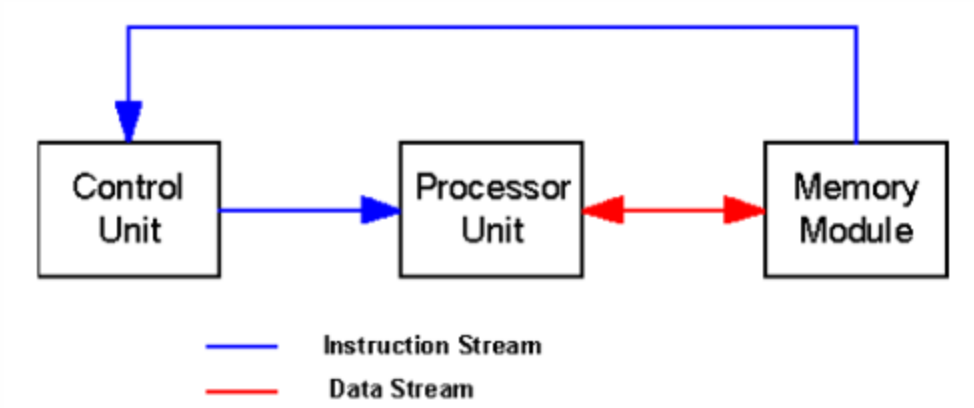
*It has **four categories: SISD, SIMD, MISD, and MIMD.***

---

1. SISD — Single Instruction, Single Data

What it is:
*SISD is a **sequential architecture** in which one processor executes one instruction stream on one data stream.*

How it works:
The processor fetches and executes instructions one after another.



Example:
Intel 8085 and traditional single-core uniprocessor systems.

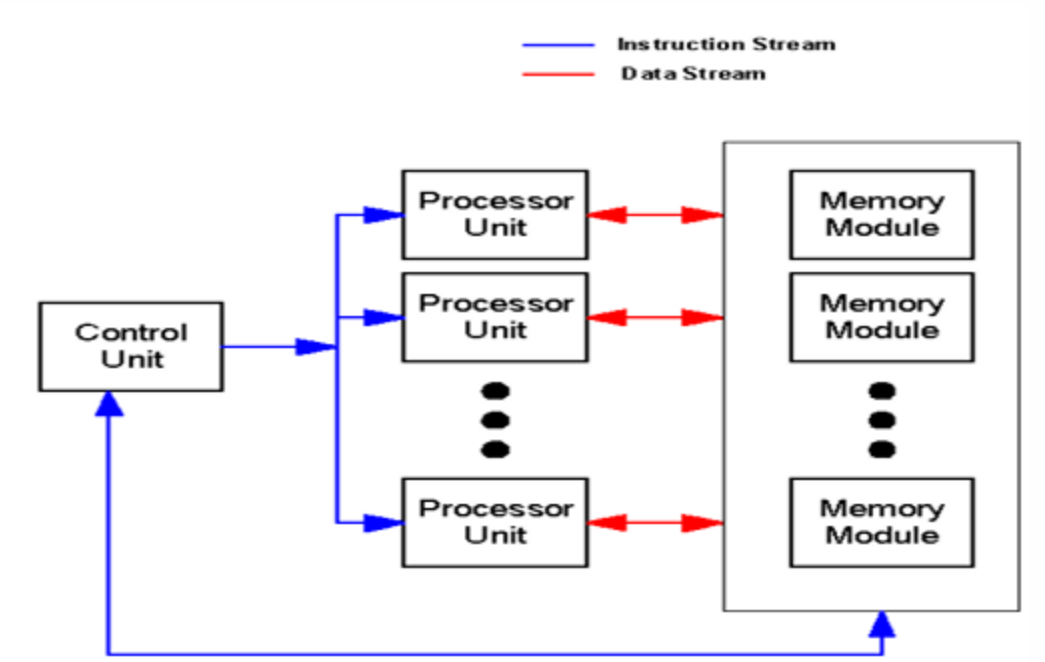
Relevance:
It provides little explicit parallelism, but modern SISD processors can still exploit ***instruction-level parallelism (ILP)*** *using techniques such as pipelining and superscalar execution.*

---

2. SIMD — Single Instruction, Multiple Data

What it is:
SIMD executes ***one instruction on multiple data elements simultaneously****.*

How it works:
A single instruction is applied to different data items at the same time, allowing the same operation to be performed in parallel.



Example:
*Cray vector processors, CPU vector instructions such as **SSE/AVX**, and GPUs.*

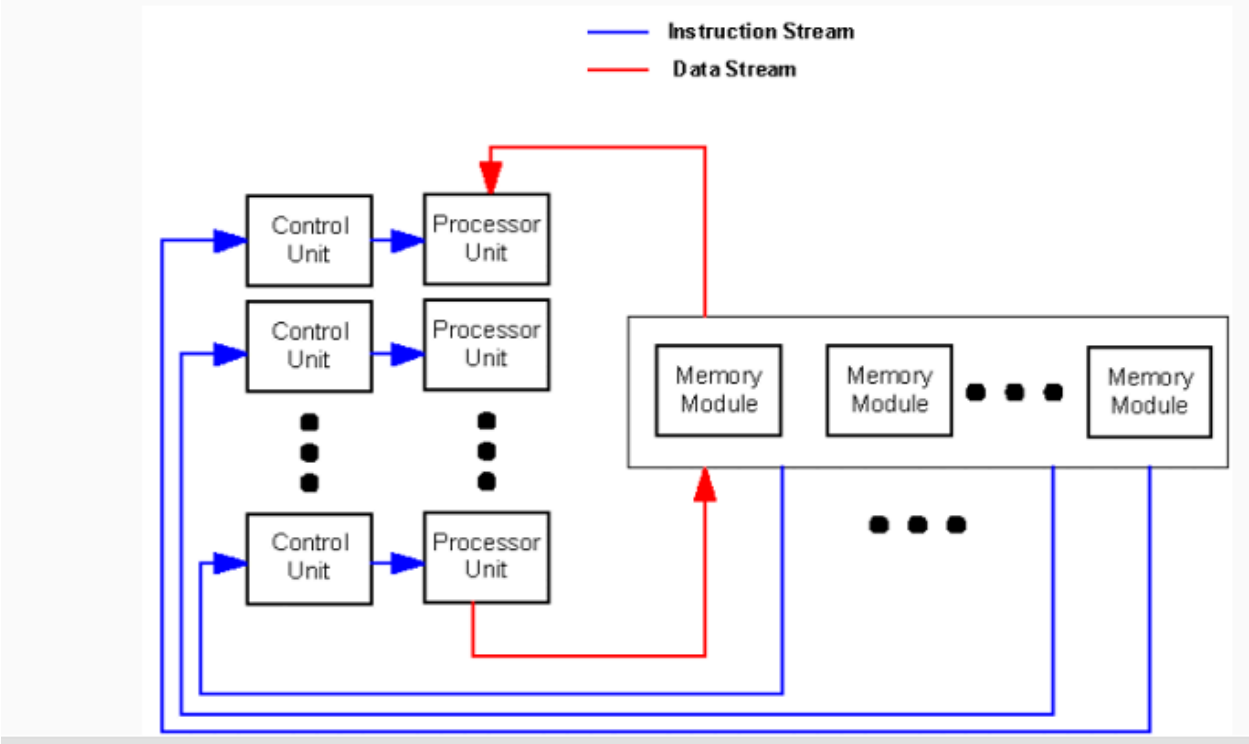
Relevance:
*SIMD is mainly used for **data-level parallelism (DLP)**. It is effective for image processing, matrix operations, scientific computation, graphics, and machine learning.*

---

3. MISD — Multiple Instruction, Single Data

What it is:
*MISD uses **multiple instruction streams operating on the same data stream**.*

How it works:
Different processing units perform different operations on the same input data.



Example:
*It is mainly found in **specialized fault-tolerant and safety-critical systems**, such as aerospace or control systems.*

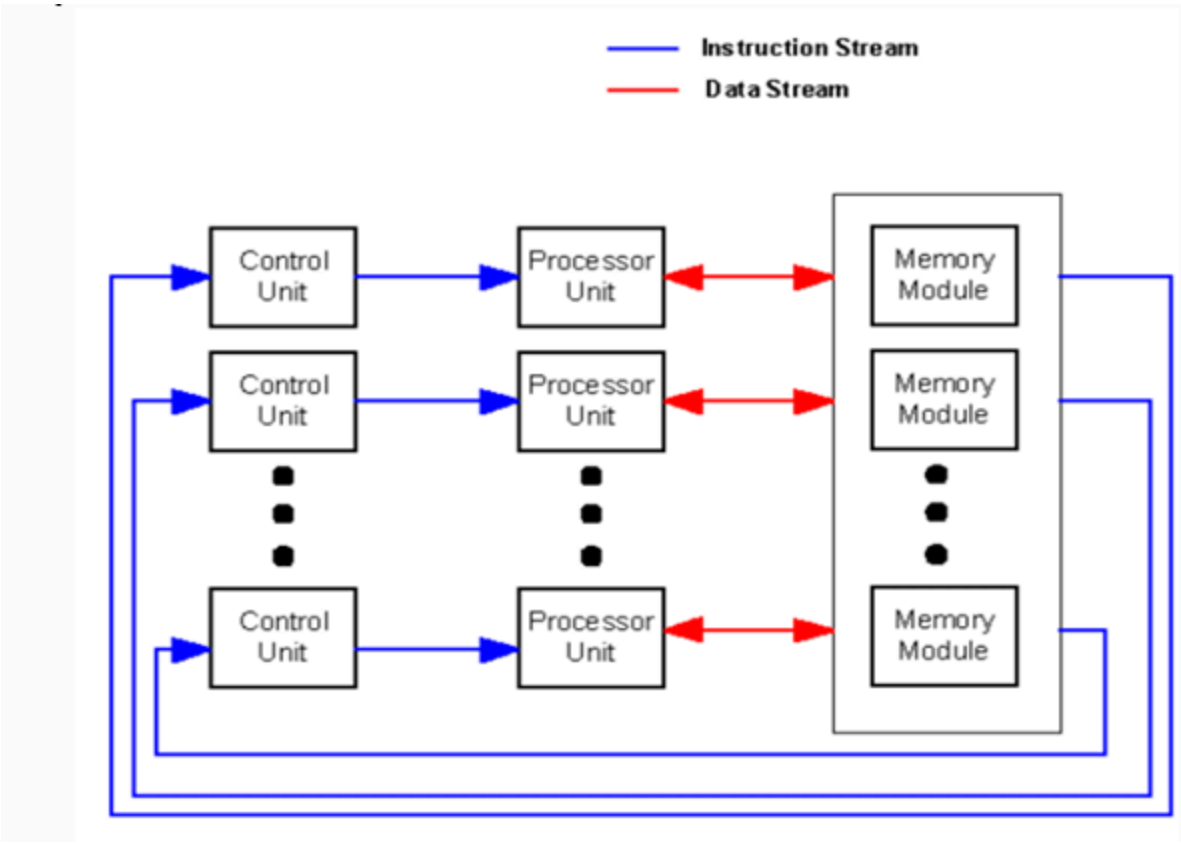
Relevance:
*MISD is **rare in general-purpose computing**. It is mainly useful where reliability, redundancy, or fault detection is important.*

---

4. MIMD — Multiple Instruction, Multiple Data

What it is:
*MIMD uses **multiple processors or processing elements**, where each can execute a different instruction stream on different data.*

How it works:
Processors operate independently and can execute different tasks simultaneously.



Examples:
Multicore CPUs such as Intel Core and AMD Ryzen, HPC clusters, and distributed computing systems.

Relevance:
*MIMD is the **most widely used architecture for general-purpose parallel computing**. It supports **task-level parallelism (TLP)** and can handle different tasks and large workloads simultaneously.*

Main types:

- ***Shared-memory:** Processors share a common memory.*
- ***Distributed-memory:** Each processor has its own local memory and communicates through a network.*

Comparison

Classification	***Instruction Streams***	***Data Streams***	***Main Parallelism***	***Example***

SISD	*1*	*1*	*Limited/ILP*	*Intel 8085*
SIMD	*1*	*Multiple*	*Data-level*	*GPU, AVX*
MISD	*Multiple*	*1*	*Specialized/redundancy*	*Fault-tolerant systems*
MIMD	*Multiple*	*Multiple*	*Task-level*	*Multicore CPU, HPC cluster*

Conclusion

*Flynn's Taxonomy provides a simple way to understand computer architectures according to their **instruction and data streams**. **SISD** is mainly sequential, **SIMD** exploits data-level parallelism, **MISD** is mainly used in specialized applications, and **MIMD** provides flexible and scalable parallel processing and is therefore dominant in modern parallel computing.*

- Compare SIMD and MIMD architectures in terms of design and applications. *(2025 Fall New)*
- Explain how parallel processing improves performance compared to sequential processing with appropriate examples. *(2025 Fall New)*

1.3 Need of parallel programming, characteristics of parallel algorithms

- What is the need of parallel programming? Explain the required characteristics of parallel algorithms. *(2025 Fall)*

Need for Parallel Programming

*Parallel programming is needed to **solve large and complex problems efficiently by using multiple processors or cores simultaneously**.*

- ***Performance Improvement:** It splits a problem into smaller tasks and distributes them among multiple processors, **allowing** faster execution.*
- ***Scalability for Large Data Sets:** It distributes large workloads across multiple processing resources, **allowing** large amounts of data to be processed efficiently.*
- ***Real-Time Processing:** It processes tasks concurrently, **ensuring** timely results and faster decision-making.*

- ***Efficient Resource Utilization:*** *It uses available CPU cores, memory, and other hardware resources effectively,* ***increasing*** *overall resource utilization.*
- ***Enabling Advanced Technologies:*** *It is mostly used in* ***AI and machine learning, GPU computing, graphics rendering, and scientific simulations****,* ***allowing*** *complex computations to be performed efficiently.*

Required Characteristics of Parallel Algorithms

A good parallel algorithm should have the following characteristics:

1. Decomposition

It divides a large problem into smaller tasks.

It helps *distribute work among processors,* ***allowing*** *tasks to execute in parallel.*

2. Concurrency

It provides multiple independent tasks that can execute simultaneously.

It helps *keep processors busy,* ***allowing*** *more work to be completed at the same time.*

3. Load Balancing

It distributes the workload evenly among processors.

It helps *prevent some processors from being overloaded while others remain idle,* ***ensuring*** *better resource utilization.*

4. Low Communication

It minimizes data exchange between processors.

It helps *reduce communication delays,* ***allowing*** *more time for useful computation.*

5. Synchronization

It coordinates the execution of parallel tasks when required.

It helps *prevent race conditions and inconsistent results,* ***ensuring*** *correct execution.*

6. Scalability

It should perform efficiently when the number of processors or problem size increases.

It helps *the algorithm take advantage of additional processing resources,* ***allowing*** *performance to scale.*

7. Low Overhead

It minimizes the extra cost of task creation, communication, and synchronization.

It helps *ensure that the performance gain from parallelism is greater than the overhead.*

8. High Speedup and Efficiency

A good parallel algorithm should achieve high ***speedup*** *and* ***processor efficiency****.*

- ***Speedup:*** $(S = T_s/T_p)$
- ***Efficiency:*** $(E = S/p)$

They help *measure how effectively multiple processors are being utilized.*

Conclusion

Thus, parallel programming is mainly used to ***improve performance, process large workloads, and utilize multiple hardware resources effectively****. A good parallel algorithm should provide* ***decomposition, concurrency, load balancing, low communication, synchronization, scalability, low overhead, and high speedup and efficiency****,* ***ensuring*** *effective use of parallel processing resources.*

1.4 Parallel Programming Techniques

- Explicit vs. implicit parallel programming *(Differentiate, 2025 Fall)*

In parallel computing, developers and systems use two primary approaches to expose and exploit parallelism: ***implicit*** *and* ***explicit parallel programming****. While implicit parallel programming relies on the compiler and operating system to automatically uncover and manage concurrency, explicit parallel programming places the responsibility of parallel execution design directly on the programmer.*

Implicit vs. Explicit Parallel Programming

Feature	***Implicit Parallel Programming***	***Explicit Parallel Programming***
Primary Responsibility	*The compiler and operating system.*	*The programmer.*
Source Language	*Conventional sequential languages (e.g., standard C, C++, Fortran, Pascal).*	*Concurrent programming languages (e.g., Chapel, Occam) or sequential languages extended with parallel libraries/directives (e.g., OpenMP, MPI).*
Compiler Role	***Parallelizing / Vectorizing Compiler:*** *detects independent operations, and automatically generates parallel object code.*	***Concurrency-Preserving Compiler:*** *Preserves programmer-defined parallelism and maps it onto machine resources.*
Programmer Effort	*Low; the programmer does not need to understand the underlying parallel architecture or parallel semantics.*	*High;designs parallel algorithms and manages synchronization and communication.*
Resource Allocation	*Handled by OS, hardware, or compiler.*	*Guided explicitly by programmer directives or library parameters.*
Examples	*Multi-threaded applications using parallelizing compilers, loop vectorization.*	*MPI (Message Passing Interface), OpenMP, PThreads, CUDA, and Chapel programming language.*

- Explicit and implicit parallel programming *(Short Note, 2025 Fall Mock & 2025 Fall New)*

1.5 Parallel Programming Models

- What is the message passing programming? In which case is it suitable? Explain. *(2025 Fall)*
- In which architecture is the message passing programming model used? Compare the message passing programming model with the shared memory programming model. *(2024 Fall)*

The message passing programming model is used in distributed memory parallel architectures. These systems are also referred to as loosely coupled architectures or multicomputers, where there is no physical sharing of memory between processing nodes. Instead, each node operates as a complete computer building block, containing its own private processor, local memory, and I/O system

Feature	Shared Memory Model (e.g., OpenMP)	Message Passing Model (e.g., MPI)
Target Architecture	Shared memory systems (SMP)	Distributed memory systems (Clusters, MPPs)
Address Space	Unified, global address space	Divided, private local address spaces
Communication	Implicit (direct memory read/write)	Explicit (`send`/`receive` library calls)
Processes/Threads	Thread-based execution (via `fork`/`join`)	Process-based execution
Data Partitioning	Managed automatically in main memory	Programmed explicitly by the developer
Synchronization	Manual locks, semaphores, and barriers	Built into synchronous message transfers

Scalability	Limited by bus/memory bottlenecks	High; scales to thousands of nodes
Complexity	Low to moderate; simple annotations	High; complex manual coordination

The message passing model is highly suitable and preferred in the following scenarios:

1. ***Distributed-Memory Architectures:***
   *Suitable for **PC clusters, distributed-memory multicomputers, and supercomputers** where processors do not share common main memory.*
2. ***Massively Scalable Applications:***
   *Suitable for applications requiring **thousands or millions of parallel processes**, especially in **HPC and big data analytics**.*
3. ***Domain Decomposition:***
   *Suitable for **data-intensive problems** where data is divided into subsets, such as **weather forecasting, matrix multiplication, and fluid dynamics**.*
4. ***Hybrid Implementations:***
   *Suitable for **multi-core, multi-node clusters**, where **MPI** handles inter-node communication and **OpenMP** handles intra-node execution.*

1.6 Parallel Algorithms for Multiprocessors

- Synchronized vs. asynchronized parallel algorithms *(Differentiate, 2025 Fall)*
- Explain the Synchronized and Asynchronized Parallel Algorithms for Multiprocessors. *(2025 Fall Mock)*

Synchronized vs. Asynchronized Parallel Algorithms

*Parallel algorithms are broadly classified into **Synchronized (Synchronous)** and **Asynchronized (Asynchronous) Parallel Algorithms** based on how processors coordinate their work and manage data dependencies.*

1. Synchronized Parallel Algorithms

*In a **synchronized parallel algorithm**, processors coordinate their progress at predefined **synchronization barriers**.*

- ***Step-by-Step Execution:*** *Processors complete step (i) before proceeding to step (i+1).*
- ***Strict Determinism:*** *Provides predictable data access and simpler design and debugging.*
- ***Communication:*** *Uses shared variables, locks, semaphores, or tightly coupled networks.*
- ***Example: Parallel Binary Summation*** *and* ***Cannon's Algorithm****.*
- ***Pros:*** *Predictable, deterministic, and avoids race conditions.*
- ***Cons:*** *Synchronization overhead and* ***load imbalance****; faster processors may wait for slower processors.*

2. Asynchronized Parallel Algorithms

In an ***asynchronized parallel algorithm****, processors operate independently without global execution barriers.*

- ***Independent Task Progress:*** *Each processor executes tasks at its own pace.*
- ***Latency Hiding:*** *Execution continues during slow memory or network operations.*
- ***Asynchronous Communication:*** *Uses buffered, non-blocking message-passing.*
- ***Examples: Pure Dataflow Computing, non-blocking MPI, and Dynamic Work-Stealing****.*
- ***Pros:*** *Maximizes hardware utilization, throughput, and scalability.*
- ***Cons:*** *Complex to design and susceptible to* ***race conditions*** *and* ***deadlocks****.*

1.7 Performance of Parallel Algorithms

- In the example of adding 'n' numbers using 'p' processing elements, can granularity be achieved in a cost-optimal manner? Provide justification for your answer. **OR** Is it possible to implicitly reduce the size of an instruction? Explain your reasoning. *(CA / Old)*

Chapter 2: Memory Technology

2.1 Hierarchical Memory Structures & Locality

- What is memory hierarchy? How do the different hierarchies of memory help to enhance the performance of the computer? Explain. *(2025 Fall)*

What is Memory Hierarchy? How do the different hierarchies of memory help to enhance the performance of the computer? Explain.

What is a Memory Hierarchy?

*A **memory hierarchy** is an organized structure of different memory types arranged in several levels to balance **performance, capacity, and cost**.*

As we move further from the processor:

- ***Memory capacity increases.***
- ***Access time increases** (speed decreases).*
- ***Cost per byte decreases.***
- ***Transfer bandwidth decreases.***
- ***Unit of transfer size increases.***

Levels of Memory Hierarchy

1. ***Register Memory:** Fastest, smallest, and located inside the CPU. It holds operands and instructions currently being processed.*
2. ***Cache Memory (L1, L2, L3):** Small, high-speed **SRAM** that stores frequently accessed data and instructions.*
3. ***Main Memory / RAM:** Usually **DRAM**, which stores active programs and data during execution.*
4. ***Secondary Storage: SSDs and magnetic hard disks** provide large, non-volatile storage but are slower than RAM.*
5. ***Backup Storage: Magnetic tapes or optical disks** are mainly used for off-line archival storage.*

How Memory Hierarchy Enhances Computer Performance

1. ***Cushioning the Memory Wall:***
   *Caches reduce the speed gap between the fast processor and slower main memory, reducing processor **stall time**.*
2. ***Exploiting Principle of Locality:***
   Memory hierarchy keeps frequently used data in faster levels using:
   - ***Temporal Locality:** Recently accessed data is likely to be accessed again.*
   - ***Spatial Locality:** Nearby memory locations are likely to be accessed.*
   - ***Sequential Locality:** Program execution generally follows sequential order.*
3. ***Inclusion Property:***
   *Data in an upper level is maintained as a subset of the next lower level. A **cache miss** retrieves the required block from the lower level.*
4. ***Virtual Memory and Demand Paging:***
   ***Virtual Memory** uses secondary storage to logically expand RAM. **Demand paging** loads only active pages into physical RAM, supporting efficient multitasking.*
5. ***Translation Lookaside Buffer (TLB):***
   *A **TLB** is a specialized cache that stores recently used virtual-to-physical address translations, reducing the overhead of address translation.*

Conclusion

*Thus, memory hierarchy provides **high speed at low cost** by combining small, fast memory with large, slower memory. It improves overall computer performance by reducing access time, memory stalls, and effective memory cost.*

- Locality of reference *(Short Note, 2025 Fall Mock)*
- What new memory technologies can be used to improve hierarchical memory systems and make computational tasks faster and more efficient? Explain. *(CA / Old)*

2.2 Memory Capacity Planning

- What are the factors to be considered during memory capacity planning? Explain. *(2025 Fall Mock)*

What are the factors to be considered during memory capacity planning? Explain.

***Memory capacity planning** is the process of forecasting, analyzing, and allocating adequate memory resources to ensure optimal **performance, reliability, and scalability** of a system or application.*

1. Workload Requirements

- ***Application Type:** Different workloads such as databases, web servers, and AI/ML models require different memory.*
- ***Concurrency Levels:** The number of simultaneous processes, threads, or users directly affects memory requirements.*
- ***Usage Patterns:** Memory should be planned according to **average and peak usage** to handle workload spikes.*

2. System Architecture

- ***Hardware Configuration:** CPU-to-memory bandwidth, memory hierarchy latency, and network speeds affect memory utilization.*
- ***Software Optimization:** Inefficient code, unoptimized data structures, and memory leaks can increase memory requirements.*
- ***Operating System: Virtual memory, demand paging, and swapping** affect physical RAM utilization.*

3. Future Scalability

- ***Growth Projections:*** *Planning should consider future increases in users, transactions, and data volume.*
- ***New Features:*** *New software functionalities may require additional memory.*
- ***Elasticity:*** *The system should dynamically adapt and scale to changing workload variations.*

4. Cost-Performance Trade-offs

Memory capacity should balance ***access speed, capacity, and cost****. Fast memory such as* ***SRAM cache*** *is expensive, while slower memory such as* ***DRAM and secondary storage*** *is cheaper. Therefore, memory hierarchy should be optimized according to performance and budget.*

5. Performance and Resource Analysis

Workload profiling, predictive modeling, resource pooling, benchmarking, and stress testing *help identify memory requirements, bottlenecks, and future capacity needs.*

Conclusion

Therefore, memory capacity planning considers ***workload requirements, system architecture, future scalability, cost-performance trade-offs, and performance analysis*** *to ensure efficient and reliable memory utilization.*

- Memory capacity planning *(Short Note, 2025 Fall New)*

2.3 Cache Memories and Management

- What is cache memory? Explain the different cache replacement algorithms *(2025 Fall)*

Cache memory *is a small, extremely high-speed,* ***SRAM-based*** *storage layer situated close to or directly on the CPU. It acts as a buffer between the CPU and slower* ***main memory (DRAM)****.*

Purpose: *It minimizes* ***data access latency****, reduces memory bottlenecks, and improves CPU processing efficiency.*

- ***Cache Hit:*** *Requested data is found in the cache and delivered quickly.*
- ***Cache Miss:*** *Data is not found, so a block is fetched from main memory and loaded into the cache.*
- ***Principle of Locality:*** *Cache works mainly through:*
    - ***Temporal Locality:*** *Recently accessed data is likely to be accessed again.*
    - ***Spatial Locality:*** *Nearby memory locations are likely to be accessed soon.*

Cache Replacement Algorithms

*When the cache is full and a new block needs to be loaded, a replacement algorithm decides which block to **evict**.*

1. First-In, First-Out (FIFO)

*FIFO uses a **first-in, first-out queue**. The block that entered the cache first is evicted first, regardless of how recently it was accessed.*

Example:
*If the cache contains **A, B, C** and **D** arrives, FIFO removes **A** because it is the oldest block.*

***Effectiveness:** Simple and cheap to implement, but may evict frequently used blocks. It can also suffer from **Belady's Anomaly**.*

2. Least Recently Used (LRU)

*LRU relies on the **Principle of Locality**. It removes the block that has **not been accessed for the longest period of time**.*

Example:
*If the access order is **A, B, C, A** and the cache is full, **B** is the least recently used and may be evicted.*

***Effectiveness:** Generally provides a high **hit ratio** because it preserves recently used data. However, tracking access order has higher hardware complexity.*

3. Least Frequently Used (LFU)

*LFU removes the block with the **lowest cumulative access count**. It maintains a frequency counter for each block.*

Example:
*If the cache contains **A(5), B(2), C(1)** and a new block arrives, LFU removes **C** because it has the lowest frequency.*

***Effectiveness:** Works well for repetitive access patterns but may suffer from **frequency accumulation/cache pollution**, where an old block with a high count remains in the cache even when it is no longer used.*

4. Optimal (OPT / MIN)

- *Replaces the block that will **not be used for the longest period in the future**.*
- *Provides the **lowest possible miss rate**.*
- *Impossible to implement in real hardware because future references are unknown.*

- *Used mainly as an **analytical benchmark**.*

5. Random Replacement

- *Randomly selects a block for eviction.*
- *Simple and has low state-tracking overhead.*
- *Generally provides lower hit rates because it ignores program locality.*

6. Circular FIFO (Second-Chance / Clock)

- *Uses a **circular queue** with a reference bit.*
- *A block with reference bit `1` gets a **second chance** and the bit is reset.*
- *The first block with reference bit `0` is evicted.*
- *Approximates **LRU** with lower complexity.*

7. Optimum Cache Partition (OCP)

- *A **compiler-assisted cache replacement policy** used for **EPIC architectures**.*
- *Compiler analyzes memory references and logically partitions the cache.*
- *Uses **cache hints** to control cache staying time and reduce **cache pollution**.*

Conclusion

*Cache replacement algorithms determine which block should be removed when the cache is full. **LRU** provides good performance, **FIFO** is simple, **LFU** considers frequency, **OPT** gives the theoretical best result, while **Random, Second-Chance, and OCP** provide alternative approaches for different system requirements.*

- Direct vs. associative mapping *(Differentiate, 2025 Fall)*

Direct Mapping vs. Associative Mapping vs. Set-Associative Mapping

Feature	***Direct Mapping***	***Associative Mapping***	***Set-Associative Mapping***
Block Placement	*Exactly one specific cache block frame.*	*Any available block frame.*	*Any block frame within a specific set.*

Address Fields	*Tag, Index, Word Offset.*	*Tag, Word Offset.*	*Tag, Set Index, Word Offset.*
Search	*One tag comparison.*	*Compares tag with all block tags in parallel.*	*Compares tag with (k) tags within the set.*
Replacement Choice	***No choice**; incoming block replaces the existing block.*	***Full choice**; any block can be evicted.*	***Restricted choice**; block is selected within the set.*
Hardware Complexity	***Low***	***High***	***Moderate***
Conflict Misses	***High***	***Lowest***	***Moderate***
Performance	*Fast but may have more conflict misses.*	*High hit ratio but expensive.*	*Good balance of cost and performance.*

1. Direct Mapping

- *Every main memory block maps to **exactly one specific cache block frame**.*
- *Uses **Tag, Index, and Word Offset**.*
- *It is simple, fast, and has **low hardware cost**.*
- *It can suffer from **conflict misses** when multiple blocks map to the same cache location.*
- ***No replacement policy is required.***

2. Associative Mapping (Fully Associative)

- *Any main memory block can be placed in **any cache block frame**.*
- *Uses **Tag and Word Offset**.*
- *Searches all cache tags in parallel using **associative search/CAM**.*
- *Provides maximum flexibility and the **lowest conflict misses**.*
- *Hardware is complex and expensive.*

3. Set-Associative Mapping

- *Cache block frames are divided into **sets**.*
- *A memory block maps to one specific set but can occupy **any frame within that set**.*
- *Uses **Tag, Set Index, and Word Offset**.*

- *Provides a balance between **direct and associative mapping**.*
- *Has moderate hardware complexity and reduces conflict misses compared with direct mapping.*

Conclusion

***Direct mapping** is simplest and cheapest, **associative mapping** provides maximum flexibility and hit ratio, while **set-associative mapping** provides a balance between performance, cost, and complexity.*

- How do FIFO, LRU, and LFU cache replacement policies differ in terms of their approaches and effectiveness? Discuss with examples. *(2024 Fall)*

Cache replacement policies determine which cache block is removed when the cache is full and a new block needs to be loaded.

Feature	***FIFO***	***LRU***	***LFU***
Approach	*Evicts the **oldest block**.*	*Evicts the block **not accessed for the longest time**.*	*Evicts the block with the **lowest access count**.*
Logic	*Arrival order.*	*Temporal locality.*	*Access frequency.*
Complexity	***Low***	***High***	***High***
Effectiveness	*Suboptimal; ignores locality.*	***Excellent**; generally gives a high hit ratio.*	*Moderate; effective for repetitive workloads.*
Limitation	*Can suffer from **Belady's Anomaly**.*	*High overhead for tracking access order.*	***Cache pollution** due to frequency accumulation.*

1. First-In, First-Out (FIFO)

*FIFO uses a **first-in, first-out queue**. The block that entered the cache first is evicted first, regardless of how recently it was accessed.*

Example:
*If the cache contains **A, B, C** and **D** arrives, FIFO removes **A** because it is the oldest block.*

***Effectiveness:** Simple and cheap to implement, but may evict frequently used blocks. It can also suffer from **Belady's Anomaly**.*

2. Least Recently Used (LRU)

*LRU relies on the **Principle of Locality**. It removes the block that has **not been accessed for the longest period of time**.*

Example:
*If the access order is **A, B, C, A** and the cache is full, **B** is the least recently used and may be evicted.*

***Effectiveness:** Generally provides a high **hit ratio** because it preserves recently used data. However, tracking access order has higher hardware complexity.*

3. Least Frequently Used (LFU)

*LFU removes the block with the **lowest cumulative access count**. It maintains a frequency counter for each block.*

Example:
*If the cache contains **A(5), B(2), C(1)** and a new block arrives, LFU removes **C** because it has the lowest frequency.*

***Effectiveness:** Works well for repetitive access patterns but may suffer from **frequency accumulation/cache pollution**, where an old block with a high count remains in the cache even when it is no longer used.*

Conclusion

***FIFO** is simple but less effective, **LRU** is generally more effective because it exploits temporal locality, while **LFU** is useful for frequently accessed data but can suffer from cache pollution.*

- Describe three cache replacement policies with appropriate examples. *(2025 Fall New)*

2.4 Virtual Memory Technology & Page Replacement

- Belady's Anomaly *(Short Note, 2025 Fall)*

Belady's Anomaly

Belady's Anomaly *is a phenomenon in virtual memory where* ***increasing the number of physical page frames results in an increase in the number of page faults*** *for a particular page reference sequence.*

Why Does It Occur?

- *It mainly occurs with the* ***FIFO (First-In, First-Out)*** *page replacement algorithm.*
- *FIFO replaces pages based on their* ***arrival order****, without considering locality or frequency of use.*
- *Adding more frames can change the replacement order and cause additional* ***page faults****.*

Example

For the reference string:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Number of Frames	***Page Faults***
3 Frames	***9***
4 Frames	***10***

Thus, increasing memory from ***3 to 4 frames*** *increases page faults from* ***9 to 10****.*

Important Point

- ***FIFO*** *→ Can suffer from Belady's Anomaly.*
- ***LRU and OPT*** *→ Do* ***not*** *suffer from Belady's Anomaly because they are* ***stack algorithms*** *with the* ***inclusion property****.*

In short: *Belady's Anomaly means* ***more memory can sometimes cause more page faults****, mainly in FIFO.*

- Virtual memory *(Short Note, 2024 Fall)*
- What is page replacement and why is it essential? Compare the optimal page replacement algorithm with the FIFO algorithm using suitable examples. *(2024 Fall)*

What is Page Replacement and Why is it Essential? Compare the Optimal Page Replacement Algorithm with the FIFO Algorithm using suitable examples.

What is Page Replacement?

***Page replacement** is the process of removing a page from physical **RAM** and replacing it with a newly requested page when all available page frames are occupied.*

Why is Page Replacement Essential?

- ***Overcomes Physical RAM Limits:** Allows programs to use a virtual address space larger than physical RAM.*
- ***Enables Multiprogramming:** Allows more processes to run by loading only required pages using **demand paging**.*
- ***Handles Page Faults:** When a required page is not in RAM, a **page fault** occurs and an existing page must be replaced.*
- ***Improves Performance:** An efficient replacement policy minimizes expensive **page faults** and improves memory utilization.*

Comparison of FIFO and Optimal Page Replacement

Feature	***FIFO***	***Optimal (OPT)***
Approach	*Replaces the page that has been in memory the longest.*	*Replaces the page that will not be used for the longest time in the future.*
Basis	*Arrival order.*	*Future memory references.*
Implementation	*Simple queue.*	*Cannot be practically implemented.*
Performance	*May produce more page faults.*	*Produces the **minimum possible page faults**.*
Limitation	*Can suffer from **Belady's Anomaly**.*	*Requires knowledge of future references.*

Use	*Practical page replacement method.*	*Theoretical benchmark.*

Example

Consider the page reference string:

[
0,1,2,4,2,3,7,2,1,3,1
]

*with **3 page frames**.*

1. FIFO

*FIFO removes the **oldest page** when a page fault occurs.*

- *Total Page Faults = **9***
- *Total Hits = **2***
- *Hit Ratio = **2/11 ≈ 18.1%***

***Example:** When page **4** is requested, page **0** is removed because it entered memory first.*

2. Optimal (OPT)

*OPT removes the page that will be **used farthest in the future**.*

- *Total Page Faults = **7***
- *Total Hits = **4***
- *Hit Ratio = **4/11 ≈ 36.3%***

***Example:** When page **4** is requested, page **0** is removed because it is **never used again**.*

Conclusion

***FIFO** is simple and practical but may cause more page faults and suffer from **Belady's Anomaly**. **Optimal** provides the minimum possible page faults but cannot be implemented in practice because future memory references are unknown. Therefore, OPT is mainly used as a **benchmark** for evaluating practical algorithms.*

- What is a page fault? When does it occur? How is it handled? Explain. *(2025 Fall Mock & 2025 Fall New)*

What is a Page Fault?

*A **page fault** is a type of virtual memory miss that occurs when a running program accesses a memory page that is mapped in its virtual address space but is not currently present in physical **RAM**.*

*The operating system uses **demand paging**, where pages are loaded into RAM only when they are required.*

When Does a Page Fault Occur?

A page fault occurs when:

1. ***Instruction or Data Referencing:** A program accesses an instruction or data located on a page not currently in RAM.*
2. ***Present/Valid Bit Failure:** The **MMU** or **TLB** detects that the page table entry has an invalid **present/valid bit**.*
3. ***Access Rights Violation:** A memory access violates protection rules, such as writing to a read-only page.*

How is a Page Fault Handled?

1. ***Pipeline Interruption:** The processor stops the faulting instruction and saves the **Program Counter (PC)**.*
2. ***Process Suspension:** The OS suspends the faulting process and may perform a **context switch** to another process.*
3. ***Locate the Page:** The OS locates the missing page on **secondary storage**.*
4. ***Page Replacement:** If RAM is full, the OS selects a page using a replacement policy such as **LRU, FIFO, or LFU**. A modified page is written back to disk.*
5. ***Load the Page:** The required page is loaded into a physical memory frame.*
6. ***Update Tables:** The OS updates the **page table** and invalidates stale **TLB** entries.*
7. ***Resume Execution:** The process is returned to the ready queue, and the faulting instruction is restarted using the saved **PC**.*

Conclusion

*A page fault occurs when a required page is not present in RAM. The operating system handles it by locating the page, performing page replacement if necessary, updating the **page table and TLB**, and restarting the interrupted instruction.*

- Which page replacement algorithm works best for improving memory usage and system performance? Explain. *(CA / Old)*

Least Recently Used (LRU) *is generally considered the best practical page replacement algorithm because it follows the* ***Principle of Locality*** *and reduces page faults.*

1. Optimal (OPT) Algorithm

- *Replaces the page that will* ***not be used for the longest period in the future****.*
- *It gives the* ***lowest page fault rate*** *and highest hit ratio.*
- *However, it is* ***not practically implementable*** *because future page references are unknown.*
- *Therefore, it is mainly used as a* ***theoretical benchmark****.*

2. Least Recently Used (LRU) Algorithm

- *Replaces the page that has* ***not been accessed for the longest time****.*
- *It follows* ***temporal locality****: recently used pages are likely to be used again.*
- *It keeps the active* ***working set*** *in physical memory and reduces expensive page faults.*
- *Its main limitation is* ***higher overhead*** *for tracking page access history.*
- *In practice,* ***Clock/Second-Chance and Pseudo-LRU*** *are often used as approximations.*

3. Comparison with Other Algorithms

- ***FIFO:*** *Simple, but may remove frequently used pages and can suffer from* ***Belady's Anomaly****.*
- ***LFU:*** *Uses access frequency but may suffer from* ***frequency accumulation and cache pollution****.*
- ***LRU:*** *Adapts better to program locality and generally provides better performance.*

Conclusion

OPT is best theoretically, while LRU is the best practical choice *for improving memory usage and system performance because it reduces page faults by exploiting the* ***Principle of Locality****.*

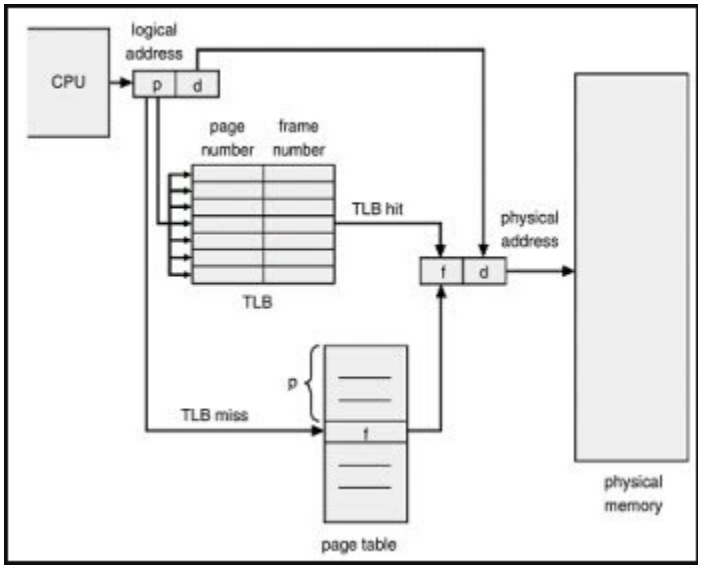
- What is translation lookaside buffer (TLB)? Explain its roles in virtual memory systems and how it improves performance. *(2025 Fall New)*

What is Translation Lookaside Buffer (TLB)? Explain its roles in virtual memory systems and how it improves performance.

What is TLB?

*A **Translation Lookaside Buffer (TLB)** is a fast hardware cache used to store recently used **virtual-to-physical address translations**. It is integrated into the **Memory Management Unit (MMU)** and helps reduce the time required for address translation.*

*A TLB entry mainly contains a **virtual page number and physical page frame number**, along with status bits such as valid, dirty, and protection bits.*



Roles of TLB in Virtual Memory

1. ***Stores Address Translations:***
   Stores recently used virtual page-to-physical page frame mappings.
2. ***TLB Lookup:***
   *The **virtual page number** is used to search the TLB.*
3. ***TLB Hit:***
   *If the translation is found, the physical page frame number is obtained quickly and the **physical address** is generated without accessing the page table.*
4. ***TLB Miss:***
   *If the translation is not found, the processor accesses the **page table** in main memory. The obtained translation is then loaded into the TLB.*
5. ***Reduces Two-Memory-Access Problem:***
   Without a TLB, virtual memory may require one memory access for the page table and another for the actual data. TLB reduces this overhead.

How TLB Improves Performance

- ***Exploits Locality:** Frequently used page translations remain in the TLB due to **temporal and spatial locality**.*

- ***Reduces Effective Access Time (EAT):*** *A high* ***TLB hit ratio*** *significantly reduces memory access time.*
- ***Removes Translation Delay:*** *TLB allows address translation to occur quickly and can operate in parallel with cache lookup.*
- ***Improves Context Switching: ASIDs (Address Space Identifiers)*** *allow translations from different processes to remain in the TLB, reducing the need for complete TLB flushing.*

Conclusion

The TLB is an important part of the ***virtual memory system****. By caching recently used address translations, it reduces page-table accesses, lowers* ***effective access time****, and improves overall system performance.*

Chapter 3: Principles of Pipelining and Vector Processing

3.1–3.6 Pipelining, Hazards & Linear/Non-linear Pipelines

- Pipelining doesn't decrease the time required for the execution of a single instruction; then how can it increase the overall throughput? Explain. *(2025 Fall)*

Pipelining doesn't decrease the time required for a single instruction; then how can it increase the overall throughput?

Pipelining increases throughput through overlapped execution of multiple instructions. *It does not reduce the* ***instruction latency*** *of an individual instruction.*

1. Instruction Latency

- ***Instruction latency*** *is the total time required to execute one instruction.*
- *Pipelining does* ***not decrease*** *this time and may slightly increase it due to pipeline overhead.*

2. Overlapped Execution

- *An instruction is divided into several stages such as* ***Fetch, Decode, Execute, and Write-Back****.*
- *Different instructions can execute in* ***different stages simultaneously****.*
- *For example, while Instruction 1 is being executed, Instruction 2 can be decoded and Instruction 3 can be fetched.*

3. Increased Throughput

- *After the pipeline is filled, **one instruction can be completed in every clock cycle**.*
- *Thus, pipelining increases the **number of instructions completed per unit time**.*

Example

*For a **4-stage pipeline**:*

- ***Non-pipelined:** 4 cycles per instruction.*
- ***Pipelined:** After the pipeline is filled, approximately **1 instruction per cycle**.*

For n instructions:

$$T_{non-pipelined} = n \times k \times \tau$$

$$T_{pipelined} = [k + (n-1)] \times \tau$$

where k = number of pipeline stages and τ = clock cycle time.

Conclusion

*Pipelining **does not make an individual instruction faster**; instead, it allows **multiple instructions to execute simultaneously in different stages**, thereby increasing **instruction throughput** and improving overall system performance.*

- What are the linear and non-linear pipelines? Explain *(2025 Fall)*

Linear and Non-Linear Pipelines

***Pipelining** is a technique of dividing a computation into several stages so that different operations can be performed **concurrently**.*

1. Linear Pipeline

*A **linear pipeline** consists of processing stages connected **sequentially**. Data enters through the first stage and moves through each stage in a fixed order until it reaches the final stage.*

Features:

- *Has **k sequential stages**:*

$$S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow \cdots \rightarrow S_k.$$

- *Data follows a **fixed path**.*
- *No feedback or bypass connections.*
- *Usually performs a **single fixed function**.*
- *Produces output from the **last stage**.*
- *Control and scheduling are relatively simple.*

Example:

$$S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4$$

For example, an arithmetic pipeline may perform **fetch → decode → execute → write-back.**

---

2. Non-Linear Pipeline

*A **non-linear pipeline** has a more complex structure with **feedforward and feedback connections** between stages. Therefore, data may follow different paths depending on the operation.*

Features:

- *Contains **feedback and/or feedforward paths**.*
- *Data may **bypass or revisit** certain stages.*
- *Can perform **different functions**.*
- *Output may come from an **intermediate or final stage**.*
- *May require **multiple reservation tables**.*
- *Scheduling is more complex because **stage conflicts/collisions** may occur.*

Example:

$$S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4$$

with additional paths such as:

$$S_3 \rightarrow S_1$$

or

$$S_1 \rightarrow S_3$$

Difference Between Linear and Non-Linear Pipeline

Basis	***Linear Pipeline***	***Non-Linear Pipeline***
Structure	*Sequential*	*Complex*
Data path	*Fixed*	*Multiple paths*
Connections	*Mainly forward*	*Feedforward and feedback*
Function	*Fixed function*	*Multiple functions*
Output	*Last stage*	*Intermediate/final stage*
Reservation table	*Usually one*	*Multiple*
Scheduling	*Simple*	*Complex*

Stage conflicts	*Less likely*	*More likely*
Flexibility	*Low*	*High*

***In short:** A **linear pipeline** follows one fixed sequence of stages, whereas a **non-linear pipeline** allows multiple paths through the stages using feedforward and feedback connections.*

- Processor pipelining *(Short Note, 2025 Fall)*
- Explain the concept of pipelining in computer architecture. How does pipelining improve the performance of a processor? Derive relation of speedup due to pipeline and discuss. *(2024 Fall)*

Concept of Pipelining in Computer Architecture

***Pipelining** is an implementation technique in which CPU hardware is divided into several **sub-operation stages** such as **Instruction Fetch, Decode, Operand Fetch, Execute, and Write-Back**. Different instructions are processed simultaneously in different stages using **interface registers/latches** between stages.*

How Pipelining Improves Performance

1. ***Overlapped Execution:** Different stages work on different instructions simultaneously.*
2. ***Higher Instruction Throughput:** After the pipeline is filled, ideally **one instruction completes every clock cycle**.*
3. ***Minimizes Processor Idle Time:** While one instruction moves to the next stage, another instruction enters the pipeline.*
4. ***Higher Clock Rate:** Dividing execution into smaller stages reduces the propagation delay of each stage.*

Speedup Relation

Let:

- n = number of instructions
- k = number of pipeline stages
- t_p = pipeline clock cycle time

Non-pipelined execution:

$$T_{non-pipeline} = nkt_p$$

Pipelined execution:

$$T_{pipeline} = (k + n - 1)t_p$$

Therefore, **speedup** is:

$$S = \frac{T_{non-pipeline}}{T_{pipeline}}$$

$$\boxed{S = \frac{nk}{k + n - 1}}$$

For $n \to \infty$:

$$\boxed{S \to k}$$

*Thus, the **maximum theoretical speedup of a (k)-stage pipeline is (k)**.*

Limitations

*Actual speedup is reduced by **pipeline register overhead, stage imbalance, and pipeline hazards** such as **structural, data (RAW, WAR, WAW), and control hazards**.*

***Conclusion:** Pipelining does not reduce individual **instruction latency**; it increases **instruction throughput** through overlapped execution.*

- Pipeline hazards *(Short Note, 2024 Fall & CA / Old)*
- What is pipelining? How can it increase the efficiency of the computer system? Explain the instruction pipelining with a typical example. *(2025 Fall Mock & 2025 Fall New)*

Instruction Pipelining: Typical Example

Consider a **4-stage instruction pipeline**:

- **FI:** Instruction Fetch
- **DA:** Decode Instruction
- **FO:** Operand Fetch
- **EX:** Execution

For **3 instructions**:

Clock Cycle	Stage 1	Stage 2	Stage 3	Stage 4
1	I_1-FI			
2	I_2-FI	I_1-DA		
3	I_3-FI	I_2-DA	I_1-FO	
4		I_3-DA	I_2-FO	I_1-EX
5			I_3-FO	I_2-EX
6				I_3-EX

- **Non-pipelined:** $3 \times 4 = 12$ clock cycles.
- **Pipelined:** $4 + (3 - 1) = 6$ clock cycles.
- After the pipeline is filled, **one instruction completes per clock cycle.**

Conclusion

*Pipelining improves computer efficiency through **overlapped instruction execution**, increasing **instruction throughput** and reducing **hardware idle time**, although it does not reduce the latency of an individual instruction.*

- What is arithmetic pipelining? With suitable examples and derivations describe how it speed up the computation? *(CA / Old)*

Arithmetic Pipelining

Arithmetic pipelining** is a technique in which a complex arithmetic operation is divided into several **sequential stages**. Each stage performs a specific sub-operation, and **interface

registers/latches *store intermediate results between stages. This allows multiple data items to be processed* ***concurrently****, increasing* ***throughput****.*

Example: Floating-Point Addition

A floating-point addition pipeline can have four stages:

1. ***Exponent comparison:*** *Find the difference between exponents.*
2. ***Mantissa alignment:*** *Shift the mantissa of the smaller exponent.*
3. ***Mantissa addition:*** *Add the aligned mantissas.*
4. ***Normalization:*** *Normalize the result and adjust the exponent.*

For example:

$$X = 0.3214 \times 10^3, \quad Y = 0.4500 \times 10^2$$

- Exponent difference: $3 - 2 = 1$
- Align Y: 0.0450×10^3
- Add: $0.3214 + 0.0450 = 0.3664$
- Result:

$$\boxed{0.3664 \times 10^3}$$

While the first result passes through all four stages, the next data item can enter Stage 1. Thus, several arithmetic operations are performed **simultaneously in different stages**.

How Arithmetic Pipelining Speeds Up Computation

Let:

- n = number of data items
- k = number of pipeline stages
- τ = pipeline clock cycle time

If stage delays are $\tau_1, \tau_2, \ldots, \tau_k$ and latch delay is d:

$$\boxed{\tau = \max(\tau_i) + d}$$

Pipelined execution time

The first data item requires k cycles, and each remaining item requires one additional cycle:

$$\boxed{T_k = (k + n - 1)\tau}$$

Non-pipelined execution time

Without pipelining, each operation requires $k\tau$:

$$\boxed{T_n = nk\tau}$$

Speedup

$$S_k = \frac{T_n}{T_k}$$

Therefore,

$$\boxed{S_k = \frac{nk}{k + n - 1}}$$

For a large number of data items:

$$\lim_{n \to \infty} S_k = k$$

Thus, the **maximum theoretical speedup of a k-stage pipeline is k.**

Example

Suppose a **4-stage arithmetic pipeline** has stage delays:

$$150,\ 120,\ 160,\ 140 \text{ ns}$$

and latch delay $= 5$ ns.

The clock period is:

$$\tau = \max(150, 120, 160, 140) + 5$$

$$\tau = 165 \text{ ns}$$

For $n = 1000$ data items:

$$T_k = (4 + 1000 - 1)(165)$$

$$\boxed{T_k = 165,495 \text{ ns} \approx 165.5\ \mu s}$$

Conclusion

*Arithmetic pipelining improves performance through **overlapped execution**, better **hardware utilization**, and increased **throughput**. It does not significantly reduce the latency of one arithmetic operation; instead, after the pipeline is filled, **one result can be produced every clock cycle**.*

- Identify the RAW, WAR, and WAW dependencies in the following instruction sequence and suggest appropriate solutions to overcome: I1: R1 = 100, I2: R1 = R2 + R4, I3: R2 = r4 - 25, I4: R4 = R1 + R3, I5: R1 = R1 + 30. *(2025 Fall New)*

3.7 Dynamic Instruction Scheduling

- Tomasulo's Algorithm *(Short Note, 2025 Fall)*
- Dynamic Instruction Scheduling *(Short Note, 2025 Fall Mock)*

Dynamic Instruction Scheduling

Dynamic instruction scheduling *is a* ***hardware-based pipeline technique*** *in which the processor dynamically detects instruction dependencies and decides the execution order* ***at run time****. Instructions may execute* ***out of order*** *when their operands and functional units are available, while the final result is normally committed in* ***program order****.*

Why is it needed?

In a traditional ***in-order pipeline****, one stalled instruction can block later independent instructions.*

Example:

DIV.D F0, F2, F4

ADD.D F10, F0, F8

SUB.D F12, F8, F14

- `DIV.D` *has a long execution time.*
- `ADD.D` *depends on the result of* `DIV.D`*, so it must wait.*
- `SUB.D` *is independent and can execute immediately.*
- ***Dynamic scheduling*** *allows* `SUB.D` *to bypass the stalled* `ADD.D` *and execute* ***out of order****.*

Thus, processor resources remain busy and ***instruction throughput increases****.*

Main Features

1. ***In-order issue:*** *Instructions are fetched/issued in program order.*
2. ***Out-of-order execution:*** *Ready instructions execute without waiting for earlier stalled instructions.*
3. ***Out-of-order completion:*** *Instructions may finish at different times.*
4. ***In-order commit:*** *Results are finally committed in program order to maintain correct program state.*

Main Techniques

1. Scoreboarding

- *Uses a* ***centralized control unit*** *called a scoreboard.*
- *Tracks functional units, registers, and instruction dependencies.*
- *Allows safe out-of-order execution.*
- *Does not use register renaming, so* ***WAR and WAW hazards*** *must be handled.*

2. Tomasulo's Algorithm

- *Uses **reservation stations** to hold waiting instructions.*
- *Uses **register renaming** to eliminate WAR and WAW hazards.*
- *Uses a **Common Data Bus (CDB)** to broadcast completed results.*
- *Allows efficient **out-of-order execution**.*

Conclusion

***Dynamic instruction scheduling** improves processor performance by allowing independent instructions to execute while dependent instructions are waiting. **Scoreboarding and Tomasulo's algorithm** are important scheduling techniques, while the **ROB** helps maintain correct program state and precise exceptions.*

- Instruction Parallelism *(Short Note, CA / Old)*

3.8–3.11 Vector Processing & Vectorization

- What is vector processing? What are the different types of vector instructions? Explain. *(2025 Fall)*

Vector Processing

***Vector processing** is a computer architecture technique in which a single instruction operates on multiple data elements of a vector (array) simultaneously. A **vector processor** uses **vector registers** and **pipelined functional units** to perform parallel operations efficiently.*

*Compared with scalar processing, vector processing reduces **loop overhead**, improves **instruction-level parallelism**, and increases **throughput**. Converting scalar code into vector operations is called **vectorization**.*

Types of Vector Instructions

1. Vector-Vector Instructions

Operate on one or two vectors element-by-element.

$$V_3 = V_1 + V_2$$

Each element of V_1 is added to the corresponding element of V_2.

2. Vector-Scalar Instructions

Operate between a vector and a single scalar value.

$$V_2 = s \times V_1$$

The scalar s is applied to every element of V_1.

3. Vector-Memory Instructions

Transfer vector data between **memory and vector registers**.

- **Vector Load:** $M \rightarrow V$
- **Vector Store:** $V \rightarrow M$

They may use **unit stride** or a fixed **stride**.

4. Vector Reduction Instructions

Combine all elements of a vector to produce a **single scalar result**.

Example:

$$s = \sum_{i=1}^{n} V_i$$

Other examples include **maximum, minimum, sum, and dot product**.

5. Gather and Scatter Instructions

- **Gather:** Collects data from **non-contiguous memory locations** into a vector register using an index vector.
- **Scatter:** Stores vector elements into **non-contiguous memory locations** using an index vector.

These are useful for **sparse data structures**.

6. Masking Instructions

Use a **vector mask register** to selectively enable or disable operations on individual vector elements.

- Mask bit **1** → operation is performed.
- Mask bit **0** → element is not modified.

This is useful for handling **conditional operations (IF conditions)** during vectorization.

Summary

Type	Main Operation	Example
Vector-Vector	Vector with vector	$V_3 = V_1 + V_2$
Vector-Scalar	Vector with scalar	$V_2 = sV_1$
Vector-Memory	Memory ↔ vector	Load/Store
Reduction	Vector → scalar	$s = \sum V_i$
Gather/Scatter	Non-contiguous memory	Indexed access
Masking	Selective elements	Mask = 1/0

In short: *Vector processing performs multiple data operations in parallel, while the six major vector instruction types provide efficient support for arithmetic, memory access, reduction, sparse data, and conditional operations.*

- Scalar vs. vector pipeline *(Differentiate, 2025 Fall)*
- What is vectorization? Discuss the challenges of vectorizing code and how a vectorizing compiler addresses these challenges.

Vectorization

Vectorization is the process of converting **scalar sequential operations** into **parallel vector operations**, where a single instruction operates on multiple data elements simultaneously. It is a

form of **SIMD (Single Instruction, Multiple Data)** parallelism. Vectorization improves **instruction throughput**, reduces loop overhead, and is useful in scientific computing, image processing, AI, and data-intensive applications.

Challenges of Vectorizing Code

1. **Control Dependence:** `IF-ELSE` statements inside loops create different execution paths and make parallel execution difficult.
2. **Loop-Carried Dependence:** When one iteration depends on the result of a previous iteration, iterations cannot execute simultaneously.
3. **Non-Unit Stride:** Non-contiguous memory access reduces memory efficiency and can increase cache and **TLB misses**.
4. **Sparse and Indirect Access:** Accesses such as `A[K[i]]` generate irregular memory addresses, making vectorization difficult.
5. **Varying Vector Length:** The loop size may not match the processor's **Maximum Vector Length (MVL)**.
6. **Complex Loop Structure:** Function calls, multiple loop exits, and I/O operations can prevent effective vectorization.

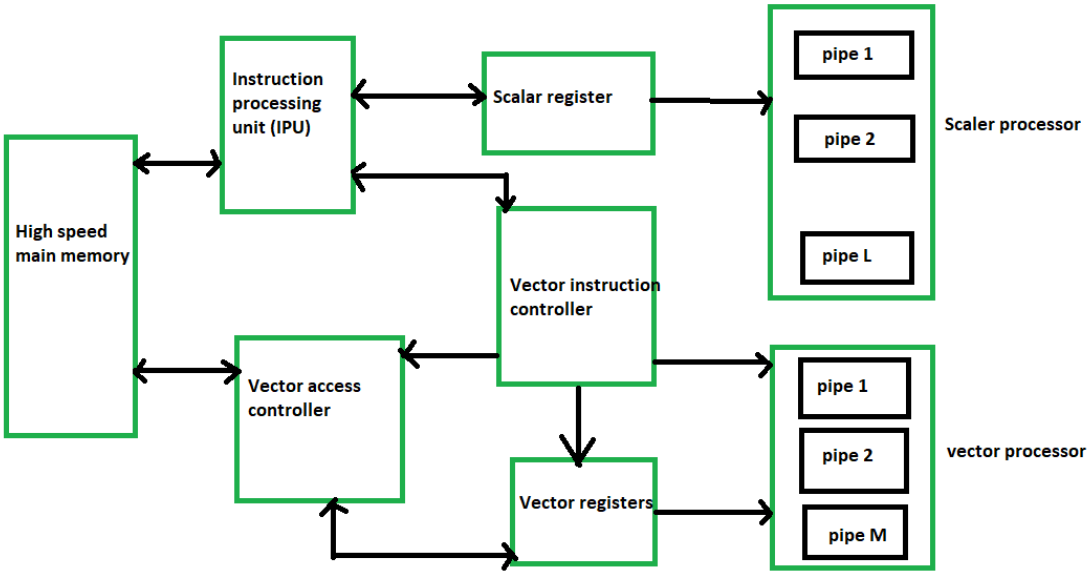
How a Vectorizing Compiler Addresses Them

Challenge	Compiler Technique
Control dependence	**If-conversion and vector masking**
Loop-carried dependence	**Dependency analysis, node splitting, temporary variables**
Scalar accumulation	**Scalar expansion and vector reduction**
Non-unit stride	**Loop interchange and strided vector instructions**
Sparse/indirect access	**Gather and scatter instructions**
Different loop length	**Strip-mining and Vector-Length Register (VLR)**
Uncertain dependencies	**Programmer directives/hints**

Conclusion

Vectorization converts sequential scalar code into **SIMD-based parallel code** to increase processor throughput. The main difficulties are **data dependencies, control dependencies, irregular memory access, and varying vector lengths**. A vectorizing compiler uses **dependency analysis, masking, loop interchange, scalar expansion, gather/scatter, and strip-mining** to overcome these limitations and generate efficient vector code.

- **OR** What are some of the use cases of vector database in advanced computing? Explain. *(2024 Fall)*
- What is vector processing? Explain the basic architecture of a vector processor with a suitable diagram. *(2025 Fall Mock & 2025 Fall New)*



- What are some common use cases of vector databases in the field of data science and AI? Explain. *(CA / Old)*

3.12 Array Processors & SIMD Algorithms

- What is array processing? Explain different types of array processors. *(2025 Fall)*

Array Processing

***Array processing** is a technique in which an **array processor** performs the same operation on multiple data elements simultaneously. It exploits **data-level parallelism** and is commonly classified under **SIMD (Single Instruction, Multiple Data)** architecture.*

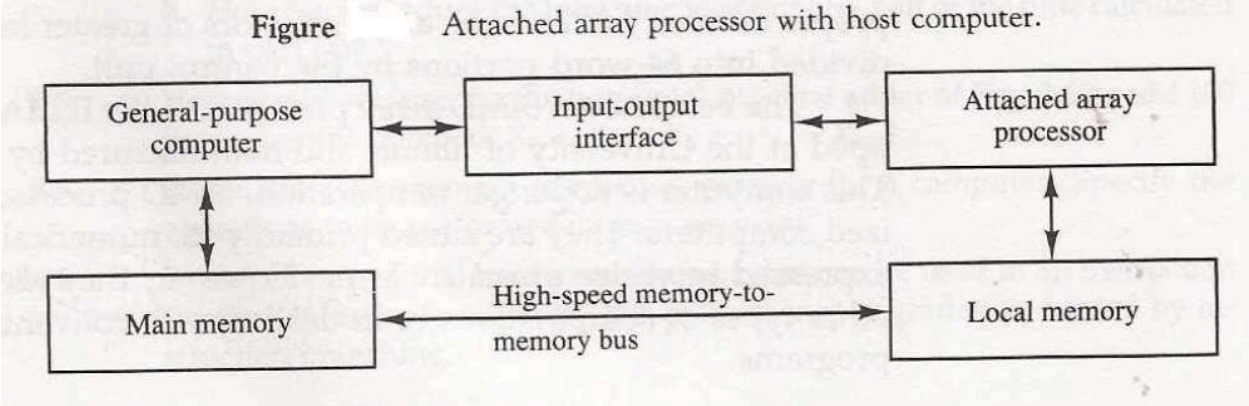
*Unlike scalar processing, which handles one data item at a time, array processing uses multiple **processing elements (PEs)** to perform operations on many array elements concurrently.*

Types of Array Processors

1. Attached Array Processor

*An **attached array processor** is a specialized accelerator connected to a **general-purpose host computer**.*

- *The host computer performs normal processing and control operations.*
- *Computationally intensive **array and matrix operations** are offloaded to the attached processor.*
- *It uses multiple **functional units** working in parallel.*
- *A high-speed **memory-to-memory bus** is generally used for communication.*



***Example:** A host computer sends a large matrix multiplication task to the attached array processor for faster computation.*

2. SIMD Array Processor

*A **SIMD array processor** contains multiple identical **Processing Elements (PEs)** controlled by a single **Control Unit (CU)**.*

- *The CU fetches and broadcasts a **single instruction** to all PEs.*
- *All PEs operate **synchronously in lockstep**.*
- *Each PE has components such as an **ALU, registers, and local memory**.*
- *A **masking mechanism** can enable or disable selected PEs.*
- *An **interconnection network** allows PEs to exchange data.*

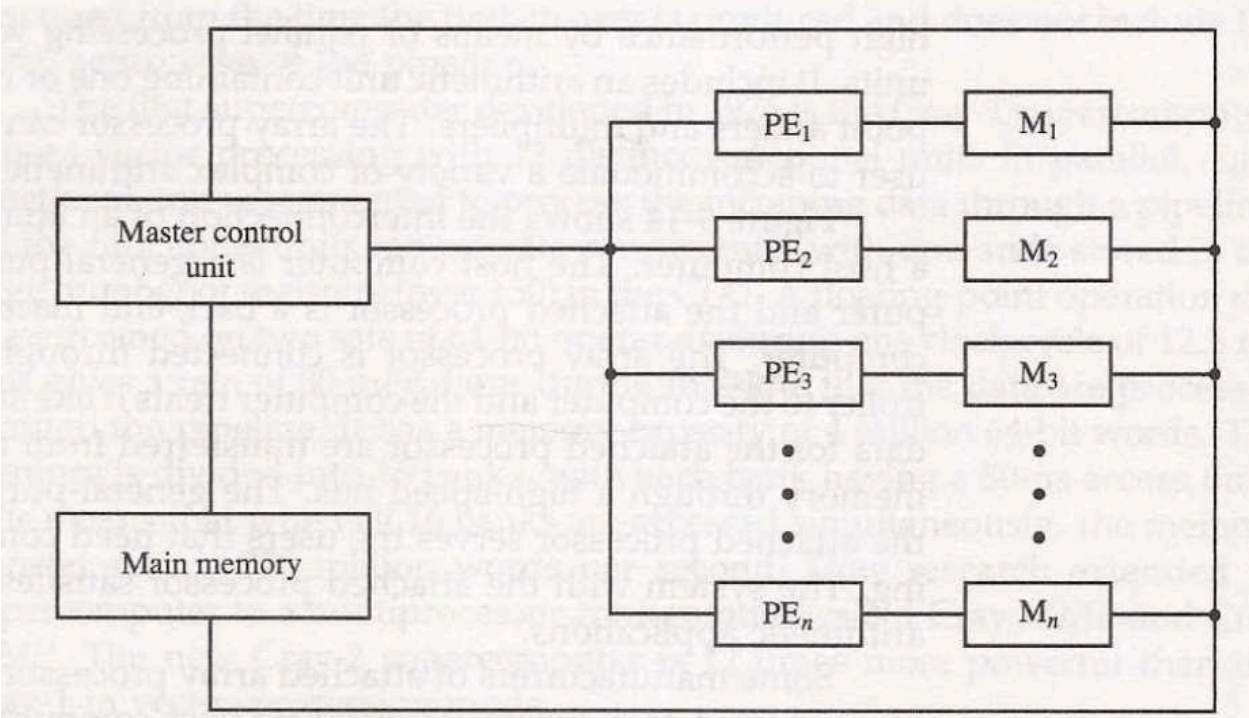


Figure SIMD array processor organization.

Example: *For*

$$C[i] = A[i] + B[i]$$

each PE can simultaneously add the corresponding elements of (A) and (B).

Comparison

Feature	***Attached Array Processor***	***SIMD Array Processor***
Relationship	*Attached to host computer*	*Independent parallel architecture*
Control	*Host computer controls it*	*Single CU controls multiple PEs*
Processing	*Offloads intensive computation*	*Parallel processing of multiple data*
Main technique	*Parallel functional units*	*Multiple PEs in lockstep*

Local memory	*Generally shared/host-connected*	*Each PE may have local memory*

Conclusion

Array processing *improves performance by exploiting* ***data parallelism****. The two major types are* ***Attached Array Processors****, which act as accelerators for a host computer, and* ***SIMD Array Processors****, which use multiple synchronized processing elements under a single control unit.*

- Array processor *(Short Note, 2024 Fall)*
- SIMD Array Processors *(Short Note, 2025 Fall Mock)*
- How can Cannon's algorithm and systolic arrays improve matrix multiplication efficiency in high-performance computing? Explain. *(CA / Old)*

Chapter 4: Multithreaded Architecture

4.1 Latency Hiding Techniques

- What is the primary purpose of latency hiding techniques in multithreaded architecture, and how do these techniques improve overall system performance? *(CA / Old)*

The ***primary purpose of latency hiding*** *is to* ***prevent the processor from remaining idle during high-latency operations*** *such as cache misses, memory accesses, I/O operations, and remote memory references.*

It achieves this by ***overlapping the waiting time of one thread with the useful execution of another ready thread****.*

How It Improves System Performance

1. ***Reduces Processor Idle Time:***
   When one thread stalls, the processor switches to another ready thread instead of waiting.
2. ***Fast Context Switching:***
   Hardware maintains multiple ***thread contexts*** *such as registers and the* ***Program Counter (PC)****, allowing rapid switching between threads.*
3. ***Increases Throughput:***
   Useful instructions from other threads fill the pipeline slots that would otherwise be wasted, increasing overall ***instruction throughput****.*

4. ***Hides Memory Latency:***
   *During a **cache miss** or slow main-memory access, another thread can execute while the required data is being fetched.*
5. ***Improves Hardware Utilization:***
   *Techniques such as **SMT** allow instructions from multiple threads to use available execution units, keeping the processor resources busy.*
6. ***Supports Large-Scale Parallel Systems:***
   *In distributed systems, computation from one thread can continue while another waits for **remote memory or network data**.*

Example

Suppose T_1 requests data from main memory and takes 100 cycles to receive it.

- **Without latency hiding:** CPU may remain idle for much of those 100 cycles.
- **With multithreading:** CPU switches to T_2 and performs useful work during those 100 cycles.

Therefore:

$$\boxed{\text{Latency Hiding} = \text{Overlap Waiting Time with Useful Computation}}$$

Conclusion: *Latency hiding does not necessarily reduce the actual latency of an operation; instead, it **reduces its impact on system performance** by keeping the processor busy, thereby improving **throughput, resource utilization, and overall system efficiency**.*

- What is latency hiding? Explain how multithreading helps in hiding latency with suitable examples. *(2025 Fall New)*

Latency Hiding

***Latency hiding** is a technique used to reduce the **performance impact of delays** by overlapping a waiting operation with useful computation. Instead of keeping the processor idle during **memory, I/O, network, or execution latency**, the processor performs other useful work.*

How Multithreading Hides Latency

***Multithreading** allows multiple threads to share processor resources. When one thread becomes **stalled** because it is waiting for data, the processor switches to another **ready thread**.*

The process is:

1. *Thread (T1) requests data from memory.*
2. *(T1) becomes stalled due to **memory latency**.*
3. *The processor switches to (T2).*
4. *(T2) performs useful computation while (T1) waits.*
5. *When the required data arrives, (T1) resumes execution.*

*Thus, the waiting time of (T1) is **overlapped with useful computation of (T2)**.*

Types of Multithreading

- ***Fine-grained multithreading:** Switches between threads frequently, often every clock cycle.*
- ***Coarse-grained multithreading:** Switches when a thread encounters a major stall such as a cache miss.*
- ***Simultaneous Multithreading (SMT):** Instructions from multiple threads can execute in the same clock cycle.*
- ***Massive multithreading:** GPUs maintain thousands of threads and switch to another ready thread when one is waiting for memory.*

Suitable Examples

1. Cache Miss:
If (T1) experiences a cache miss, the processor executes (T2) while data is fetched from main memory. Therefore, memory latency is hidden.

2. Network Access:
If (T1) waits for data from a remote computer, (T2) can perform other computations during the network delay.

3. GPU Processing:
*When one GPU **warp** stalls because of a memory access, another ready warp is scheduled. This keeps the GPU execution units busy.*

Conclusion

***Latency hiding does not necessarily reduce the actual latency; it reduces its effect on overall performance.** Multithreading achieves this by **overlapping latency with useful computation**, thereby improving **processor utilization and throughput**.*

4.2–4.3 Multithreading Principles & Architectures

- What is a multithreading architecture? Explain the principles used in multithreading architecture to increase the throughput of a computer. *(2024 Fall)*

*A **multithreading architecture** is a processor architecture that allows a single CPU core to maintain and execute **multiple threads** concurrently. Each thread has its own **hardware context**, such as **Program Counter (PC), registers, and status information**, while the processor's **execution units, caches, and memory system** are shared.*

*The main goal is to exploit **Thread-Level Parallelism (TLP)** and increase **computer throughput**.*

Principles Used to Increase Throughput

1. ***Latency Hiding***
   *When one thread stalls due to a **cache miss, memory access, or I/O operation**, another ready thread is executed. Thus, waiting time is overlapped with useful computation.*
2. ***Fast Hardware Context Switching***
   *The processor maintains multiple thread contexts in hardware. Therefore, switching between threads is very fast because registers and the **PC** do not need to be saved and restored to memory.*
3. ***Thread-Level Parallelism (TLP)***
   *Instructions from independent threads provide more opportunities for parallel execution. This helps overcome limitations caused by **data and control dependencies** within a single thread.*
4. ***Efficient Resource Utilization***
   *Multiple threads share resources such as **ALUs, floating-point units, caches, and memory buses**. When one thread cannot use a resource, another thread can utilize it, reducing hardware idle time.*
5. ***Thread Scheduling and Interleaving***
   Multithreading uses different approaches:
   - ***Fine-Grained Multithreading:** switches between threads frequently, often every clock cycle.*
   - ***Coarse-Grained Multithreading:** switches when the current thread encounters a major stall.*
   - ***Simultaneous Multithreading (SMT):** issues instructions from multiple threads in the **same clock cycle**.*

Example

Suppose Thread T_1 is waiting for data from main memory. Instead of keeping the CPU idle:

$$T_1 \rightarrow \text{Memory Stall}$$

the processor executes:

$$T_2 \rightarrow T_3 \rightarrow T_4$$

while T_1 waits. When the data becomes available, T_1 resumes.

Conclusion

Multithreading increases throughput by **overlapping the execution of multiple threads, hiding latency, reducing processor idle time, and improving utilization of shared hardware resources.**

$$\boxed{\text{Multithreading} \rightarrow \text{TLP} \rightarrow \text{Higher Throughput}}$$

- What is multithreading? How can it help to enhance the performance of a computer? Explain the issues of multithreading. *(2025 Fall Mock)*

Issues of Multithreading

1. ***Synchronization:***
   *Shared data requires **locks, mutexes, semaphores, and barriers**, which add complexity and overhead.*
2. ***Race Conditions:***
   Multiple threads accessing shared data simultaneously can produce incorrect or unpredictable results.
3. ***Deadlock:***
   Two or more threads may wait indefinitely for resources held by each other.
4. ***Starvation:***
   A thread may continuously be denied CPU time or required resources.
5. ***Resource Contention:***
   *Threads compete for shared **CPU resources, cache, memory bandwidth, and TLB**, which can reduce performance.*
6. ***False Sharing:***
   *Different variables modified by different threads may occupy the same **cache line**, causing unnecessary cache-coherence traffic.*
7. ***Debugging Complexity:***
   Non-deterministic execution makes multithreaded programs difficult to test and debug.

Conclusion

Multithreading improves computer performance by ***exploiting TLP, hiding latency, increasing throughput, and improving resource utilization****. However,* ***synchronization, race conditions, deadlocks, resource contention, false sharing, and debugging complexity*** *are major challenges.*

4.4–4.8 Cluster, Grid, and Neural Computing

- Compare the cluster computing with grid computing showing similarities as well as differences. *(2024 Fall)*

1. Cluster Computing

Cluster computing *is a system in which a group of interconnected computers, called* ***nodes****, work together as a single computing system. The nodes are usually located in the same place and connected through a high-speed network.*

2. Grid Computing

Grid computing *is a distributed computing system that combines computing resources from different locations and organizations to solve large and complex problems. The resources may be geographically distributed and heterogeneous.*

Similarities between Cluster and Grid Computing

1. ***High Performance:*** *Both are used to solve large, complex, and computationally intensive problems.*
2. ***Multiple Computers:*** *Both use multiple independent computers connected through a network.*
3. ***Parallel Processing:*** *Tasks can be divided into smaller parts and processed concurrently.*
4. ***Distributed Computing:*** *Both are based on the concept of distributed computing and resource sharing.*
5. ***Scalability:*** *Both systems can improve performance by adding additional computing nodes.*
6. ***Fault Tolerance:*** *Failure of one node can be handled by redistributing or recovering tasks, depending on the system design.*
7. ***Communication:*** *Both require communication mechanisms such as* ***message passing*** *to coordinate tasks between nodes.*

Differences between Cluster and Grid Computing

Feature	***Cluster Computing***	***Grid Computing***
Coupling	*Tightly coupled*	*Loosely coupled*
Location	*Usually in one location*	*Geographically distributed*
Hardware	*Usually homogeneous*	*Usually heterogeneous*
Operating System	*Generally similar OS*	*May use different operating systems*
Administration	*Usually under one organization*	*May involve multiple organizations*
Control	*Centralized management*	*Decentralized management*
Network	*High-speed LAN/interconnect*	*Internet/WAN or other networks*
Communication Latency	*Low*	*Relatively high*
Resource Allocation	*Dedicated and relatively static*	*Shared and dynamic*
System View	*Often works like a single system*	*Nodes remain independent*
Main Purpose	*Tightly coupled parallel applications*	*Large distributed workloads and resource sharing*
Example	*HPC cluster in a research center*	*SETI@home or a multi-university research grid*

Simple Example

Cluster:
*A university has **20 identical computers in one laboratory** connected by a high-speed network. They work together to perform a weather simulation.*

Grid:
*Universities in **Nepal, India, Japan, and the USA** contribute their available computers to a common research project. The computers work together over wide-area networks.*

Conclusion

***Cluster computing** uses a group of closely connected computers under relatively centralized control and is suitable for applications requiring **fast communication and synchronization**.*
***Grid computing** combines geographically distributed and heterogeneous resources and is suitable for **large workloads that can be divided into relatively independent tasks**.*

In short:

> ***Cluster = tightly connected computers working together at one place.***
> ***Grid = geographically distributed computers sharing resources for a common task.***

- What is cluster computing? How does it differ with grid computing? Explain. *(2025 Fall Mock)*
- Explain the key differences between cluster computing, neural computing, and grid computing. How does each approach contribute to high-performance computing, and in what scenarios would one be preferred over the others? *(CA / Old)*
- Differentiate between cluster computing and grid computing. Discuss their architecture, resource management, scalability, and one major advantage and disadvantage of each. *(2025 Fall New)*

***Cluster computing** and **grid computing** are both forms of distributed computing used to provide high computational performance. However, they differ mainly in **coupling, location, resource management, and scalability**.*

1. Cluster Computing

***Cluster computing** is a group of interconnected computers that work together as a single computing resource. The computers are usually located in the same place and connected through a high-speed network.*

Architecture

- *It is a **tightly coupled** architecture.*
- *Nodes are usually **homogeneous**, having similar hardware and software.*
- *Nodes are generally located in a single data center or organization.*
- *It commonly consists of a **master node** and multiple **compute nodes**.*
- *The master node manages jobs, while compute nodes perform calculations.*
- *High-speed networks such as InfiniBand or high-speed Ethernet are commonly used.*

Resource Management

- *Resource management is generally **centralized**.*
- *A central scheduler allocates CPU, memory, and other resources to jobs.*
- *Cluster management software such as **Slurm** can be used for scheduling.*
- *Parallel applications may use **MPI** for communication between nodes.*

Scalability

- *Clusters can be scaled by adding more nodes.*
- *However, scalability is relatively limited by physical infrastructure, network bandwidth, communication overhead, and synchronization.*
- *As the number of nodes increases, performance may eventually show diminishing returns.*

Major Advantage

Low latency and high performance:
Because the nodes are closely connected through high-speed networks, clusters are excellent for applications requiring frequent communication between nodes.

Major Disadvantage

High setup and maintenance cost:
A cluster requires dedicated hardware, high-speed networking, physical space, cooling, electricity, and system administration.

---

2. Grid Computing

***Grid computing** combines computing resources from different locations and organizations to solve large computational problems.*

Architecture

- *It is a **loosely coupled** architecture.*
- *Resources are usually **heterogeneous**, meaning they can have different hardware and operating systems.*
- *Nodes may be distributed across different cities, countries, or organizations.*
- *Each participating system can operate independently.*
- *Grid resources are connected through WANs or the Internet.*
- *A grid may include computational, data, and service/equipment resources.*

Resource Management

- *Resource management is generally **decentralized**.*
- *Different organizations can maintain their own resource policies and managers.*
- ***Resource brokers** help discover and allocate suitable resources for jobs.*
- *Grid middleware manages resource discovery, authentication, data transfer, and job execution.*

Scalability

- *Grid computing is **highly scalable**.*
- *New computers, clusters, or organizations can be added dynamically.*
- *Resources can be obtained from geographically distributed locations.*
- *However, network bandwidth and high communication latency can affect performance.*

Major Advantage

Efficient resource utilization:
Grid computing can use existing or idle computing resources from different locations, reducing the need to purchase dedicated high-performance hardware.

Major Disadvantage

High latency and management complexity:
Because resources communicate over WANs or the Internet and belong to different administrative domains, communication, security, authentication, and compatibility become more difficult.

Cluster vs Grid Computing

Feature	***Cluster Computing***	***Grid Computing***

Architecture	*Tightly coupled*	*Loosely coupled*
Nodes	*Usually homogeneous*	*Usually heterogeneous*
Location	*Usually one location*	*Geographically distributed*
Administration	*Single organization*	*Multiple organizations*
Resource Management	*Centralized*	*Decentralized*
Network	*High-speed LAN/interconnect*	*WAN/Internet*
Latency	*Low*	*Relatively high*
Scalability	*Limited/controlled*	*Highly scalable*
Resource Allocation	*Central scheduler*	*Resource brokers/distributed schedulers*
Main Advantage	*Low latency and high performance*	*Efficient sharing of existing resources*
Main Disadvantage	*High setup and maintenance cost*	*High latency and management/security complexity*
Best suited for	*Tightly coupled parallel applications*	*Loosely coupled/distributed workloads*

Conclusion

> ***Cluster computing*** *uses* ***tightly connected, usually homogeneous computers under centralized management****, making it suitable for high-performance applications requiring frequent communication.*

Grid computing *combines* ***loosely connected, heterogeneous resources across different locations and organizations****, making it suitable for large workloads that can be distributed among independent resources.*

Easy way to remember:
Cluster = one organization + one location + tight connection.
Grid = many organizations + many locations + loose connection.

- ***Combined Question:***
- ***Compare and contrast cluster computing, grid computing, and neural computing. Explain their architectures, resource management, scalability, major advantages and disadvantages, and discuss how each contributes to high-performance computing. Also explain the scenarios in which each approach would be preferred.***

Cluster computing, grid computing, and neural computing *are important approaches used to achieve high-performance computing (HPC). However, they differ in their architecture, resource management, scalability, advantages, disadvantages, and applications.*

1. Cluster Computing

Cluster computing *is a group of interconnected computers that work together as a single computing resource. The computers are usually located in the same place and connected through a high-speed network.*

Architecture

- *It is a* ***tightly coupled*** *architecture.*
- *Nodes are usually* ***homogeneous****, having similar hardware and software.*
- *Nodes are generally located in a single data center or organization.*
- *It commonly consists of a* ***master node*** *and multiple* ***compute nodes****.*
- *The master node manages jobs, while compute nodes perform calculations.*
- *High-speed networks such as InfiniBand or high-speed Ethernet are commonly used.*

Resource Management

- *Resource management is generally* ***centralized****.*
- *A central scheduler allocates CPU, memory, and other resources to jobs.*
- *Cluster management software such as* ***Slurm*** *can be used for scheduling.*
- *Parallel applications may use* ***MPI*** *for communication between nodes.*

Scalability

- *Clusters can be scaled by adding more nodes.*
- *However, scalability is relatively limited by physical infrastructure, network bandwidth, communication overhead, and synchronization.*
- *As the number of nodes increases, performance may eventually show diminishing returns.*

Major Advantage

Low latency and high performance:
Because the nodes are closely connected through high-speed networks, clusters are excellent for applications requiring frequent communication between nodes.

Major Disadvantage

High setup and maintenance cost:
A cluster requires dedicated hardware, high-speed networking, physical space, cooling, electricity, and system administration.

Contribution to HPC

Clusters provide an economical alternative to specialized supercomputers. They are widely used for ***scientific simulations, weather modeling, molecular dynamics, engineering analysis, and large database workloads****.*

---

2. Grid Computing

Grid computing *combines computing resources from different locations and organizations to solve large computational problems.*

Architecture

- *It is a* ***loosely coupled*** *architecture.*
- *Resources are usually* ***heterogeneous****, meaning they can have different hardware and operating systems.*
- *Nodes may be distributed across different cities, countries, or organizations.*
- *Each participating system can operate independently.*
- *Grid resources are connected through WANs or the Internet.*
- *A grid may include computational, data, and service/equipment resources.*

Resource Management

- *Resource management is generally **decentralized**.*
- *Different organizations can maintain their own resource policies and managers.*
- ***Resource brokers** help discover and allocate suitable resources for jobs.*
- *Grid middleware manages resource discovery, authentication, data transfer, and job execution.*

Scalability

- *Grid computing is **highly scalable**.*
- *New computers, clusters, or organizations can be added dynamically.*
- *Resources can be obtained from geographically distributed locations.*
- *However, network bandwidth and high communication latency can affect performance.*

Major Advantage

Efficient resource utilization:
Grid computing can use existing or idle computing resources from different locations, reducing the need to purchase dedicated high-performance hardware.

Major Disadvantage

High latency and management complexity:
Because resources communicate over WANs or the Internet and belong to different administrative domains, communication, security, authentication, and compatibility become more difficult.

Contribution to HPC

*Grid computing allows organizations to combine distributed resources for **large-scale scientific research, data analysis, simulations, and collaborative computing**. It is particularly effective for **embarrassingly parallel** problems where tasks can operate independently.*

---

3. Neural Computing

***Neural computing** is a computing approach inspired by the structure and operation of biological neural systems. It uses **artificial neural networks (ANNs)** consisting of interconnected computational units called neurons.*

*Modern neural computing commonly uses **GPUs, TPUs, and other AI accelerators** to perform large numbers of matrix and tensor operations in parallel.*

Architecture

A neural network generally consists of:

Input Layer → Hidden Layers → Output Layer

- *Neurons receive input values.*
- *Weights represent learned relationships between inputs and neurons.*
- *Activation functions process the weighted inputs.*
- *During training, algorithms such as backpropagation adjust the weights.*
- *GPUs/TPUs accelerate the large matrix and tensor operations involved.*

Resource Management

- *Resources are managed primarily around **parallel computation and memory bandwidth**.*
- *GPUs divide work among many processing units.*
- *Threads are organized into groups and execute mathematical operations concurrently.*
- *Large models can use **data parallelism** or **model parallelism** across multiple accelerators.*

Scalability

Neural computing can scale by:

- *Increasing the number of GPU/TPU devices.*
- *Using **data parallelism** to distribute training data.*
- *Using **model parallelism** to distribute a large model across multiple devices.*
- *Increasing available memory and computational resources.*

Advantages

1. *Extremely high performance for matrix and tensor operations.*
2. *Excellent for pattern recognition and machine learning.*
3. *Highly parallel architecture.*
4. *Suitable for large-scale AI workloads.*
5. *Can process complex and nonlinear relationships.*

Disadvantages

1. *Not suitable for every type of computation.*
2. *Requires large computational and memory resources for complex models.*
3. *Training can consume significant time and energy.*
4. *Programming and optimization can require specialized knowledge.*
5. *Performance depends heavily on memory bandwidth.*

Contribution to HPC

Neural computing has become a major component of modern HPC by accelerating ***deep learning, computer vision, natural language processing, scientific simulations, image processing, and AI-based data analysis****.*

Comparison Table

Feature	***Cluster Computing***	***Grid Computing***	***Neural Computing***
Architecture	*Tightly coupled*	*Loosely coupled*	*Highly parallel neural/accelerator architecture*
Nodes/Resources	*Usually homogeneous*	*Usually heterogeneous*	*GPUs, TPUs, CPUs, AI accelerators*
Location	*Usually one location*	*Geographically distributed*	*Usually local or distributed accelerators*
Control	*Centralized*	*Distributed*	*Mainly hardware/software controlled*
Network	*High-speed local network*	*WAN/Internet*	*High-bandwidth accelerator interconnects*
Resource Management	*Centralized scheduler*	*Distributed resource broker*	*Hardware/software parallel scheduling*
Communication	*Low latency*	*Relatively high latency*	*Very high internal data throughput*
Scalability	*Good*	*Very high*	*Very high for suitable AI workloads*

Best Workload	*Tightly coupled parallel tasks*	*Loosely coupled tasks*	*Matrix, tensor, and AI workloads*
Main Strength	*Low-latency parallel computing*	*Resource sharing*	*Massive computational parallelism*
Main Limitation	*Infrastructure cost*	*Network latency and complexity*	*Specialized workload requirement*
Typical Applications	*Weather modeling, simulations, HPC*	*Scientific collaboration, distributed analysis*	*AI, deep learning, computer vision*

When Should Each Be Preferred?

Cluster Computing

Choose a cluster when:

- *Tasks require **frequent communication** between processors.*
- *Very low communication latency is required.*
- *Resources are controlled by one organization.*
- *Example: **weather simulation or computational fluid dynamics**.*

Grid Computing

Choose a grid when:

- *Tasks can be divided into relatively **independent subtasks**.*
- *Resources are geographically distributed.*
- *Multiple organizations want to share computing resources.*
- *Example: **large scientific data analysis or Monte Carlo simulations**.*

Neural Computing

Choose neural computing when:

- *The workload involves **AI and machine learning**.*
- *Computation consists heavily of matrix/tensor operations.*
- *Massive data parallelism can be exploited.*

- *Example:* ***deep-learning model training, image recognition, and NLP****.*

Conclusion

The three approaches achieve high performance in different ways:

> ***Cluster computing*** *achieves performance through* ***tightly coupled parallel computers and low-latency communication****.*
>
> ***Grid computing*** *achieves performance by* ***combining geographically distributed resources****.*
>
> ***Neural computing*** *achieves performance through* ***massive parallelism in neural-network and matrix computations****.*

Therefore, ***clusters are preferred for tightly coupled HPC applications, grids for distributed and loosely coupled workloads, and neural computing for AI and data-parallel workloads****.*

Chapter 5: Multicore Processor Design

5.1–5.3 Core Integration, Memory Hierarchy & Cache Coherence

- What are the different levels of cache memories used in multi-core computers and why are they used? Explain. *(2024 Fall)*
- What are the essential factors to consider when designing and managing cache memory? Explain. *(2024 Fall)*

Cache memory is designed to provide ***fast access to frequently used data and instructions*** *while balancing* ***speed, capacity, cost, and hardware complexity****. The important factors are:*

1. Cache Performance

The main objective is to reduce the ***effective memory access time****.*

- ***Hit ratio:*** *Percentage of memory requests found in the cache.*
- ***Miss ratio:*** *Percentage of requests not found in the cache.*
- ***Hit time:*** *Time required to find and access data in the cache.*
- ***Miss penalty:*** *Additional time required to obtain data from lower-level memory.*

A good cache should have ***high hit ratio and low hit time****.*

2. Cache Mapping

A mapping technique determines where a main-memory block can be placed in the cache.

- ***Direct Mapping:*** *Each memory block has only one possible cache location. It is simple and fast but can cause conflict misses.*
- ***Fully Associative Mapping:*** *A block can be placed anywhere in the cache. It reduces conflicts but requires more complex and expensive hardware.*
- ***Set-Associative Mapping:*** *A block maps to a particular set but can occupy any line within that set. It provides a balance between direct and fully associative mapping.*

3. Block Size

The cache block or line size must be carefully selected.

- *Larger blocks can exploit* ***spatial locality*** *and reduce compulsory misses.*
- *Very large blocks can reduce the number of blocks in the cache and increase cache pollution.*
- *Therefore, the block size should provide a good balance between locality and cache capacity.*

4. Replacement Policy

When the cache is full, a replacement policy determines which block should be removed.

Common policies include:

- ***LRU (Least Recently Used):*** *Removes the block that has not been used for the longest time.*
- ***FIFO (First-In First-Out):*** *Removes the oldest block.*
- ***LFU (Least Frequently Used):*** *Removes the least frequently accessed block.*
- ***Random:*** *Selects a block randomly.*

The replacement policy should minimize cache misses without introducing excessive hardware complexity.

5. Write Policy

The cache must determine how modifications are transferred to lower memory.

- ***Write-Through:*** *Every cache write is immediately written to lower-level memory. It is simple but generates more memory traffic.*
- ***Write-Back:*** *Data is written only to the cache initially and transferred to lower memory when the block is replaced. It reduces memory traffic but requires a* ***dirty bit*** *and more complex control.*

- ***Write-Combining:*** *Combines several small writes before sending them to memory, improving write efficiency.*

6. Cache Organization

Cache can be organized as:

- ***Unified Cache:*** *Stores both instructions and data.*
- ***Split Cache:*** *Separates instructions and data into* ***I-cache and D-cache****.*

Split caches can allow instruction fetching and data access to occur simultaneously, improving processor performance.

7. Cache Size and Cost

Increasing cache size generally increases the chance of finding required data in the cache, but larger caches:

- *Require more chip area.*
- *Increase manufacturing cost.*
- *May have greater access latency.*
- *Consume more power.*

Therefore, cache size must be selected carefully.

8. Locality of Reference

Cache design depends heavily on program locality:

- ***Temporal locality:*** *Recently accessed data is likely to be accessed again.*
- ***Spatial locality:*** *Data near a recently accessed location is likely to be accessed soon.*

Effective cache design should exploit both types of locality.

9. Prefetching

Prefetching *loads data into the cache before the processor actually requests it.*

For example, when processing an array sequentially, the system can predict upcoming elements and load them in advance. This helps ***hide memory latency****, although incorrect prefetching can waste bandwidth and cache space.*

10. Multilevel Cache Organization

Modern processors commonly use ***L1, L2, and L3 caches****.*

CPU Core

↓

L1 → Very small and fastest

↓

L2 → Larger and slower

↓

L3 → Much larger and usually shared

↓

Main Memory → Very large but much slower

*This hierarchy balances **speed, capacity, and cost**.*

11. Cache Coherence in Multi-Core Systems

In multicore processors, different cores may have their own cache copies of the same data. Therefore, cache coherence must be maintained.

*Protocols such as **MESI** and **MOESI** ensure that cores do not use outdated copies of shared data.*

12. Compiler and Software Support

Software can improve cache performance by organizing data and computation to increase locality.

Examples include:

- ***Loop blocking***
- ***Loop tiling***
- *Proper data layout*
- *Sequential memory access*

These techniques reduce cache misses and improve overall performance.

Conclusion

*The essential factors in cache design and management are **performance, mapping technique, block size, replacement policy, write policy, cache organization, size and cost, locality,***

***prefetching, multilevel hierarchy, cache coherence, and software support**. The overall goal is to achieve **high hit rates and low access time while keeping hardware cost and complexity reasonable**.*

- What is a multicore system? Why are the different hierarchies of memories used in a multicore system? Explain different multicore architectures with reference to cache memory organization. *(2025 Fall Mock & 2025 Fall New)*

1. What is a Multicore System?

*A **multicore system** is a computer processor that contains **two or more independent processing cores on a single chip**. Each core can execute instructions independently, allowing multiple tasks or threads to execute in parallel.*

Each core generally contains its own:

- *ALU and execution units*
- *Registers*
- *L1 cache*
- *Sometimes a private L2 cache*

*The cores may also share a higher-level cache such as **L3** and the main memory.*

***Example:** A quad-core processor has **four independent processing cores**, so it can execute multiple threads concurrently.*

---

2. Why Are Different Memory Hierarchies Used?

*A multicore system uses several levels of memory because no single memory technology can provide **high speed, large capacity, and low cost simultaneously**.*

1. Speed Difference

Processors operate much faster than main memory. If the CPU had to access DRAM for every instruction and data request, the cores would spend considerable time waiting.

Therefore, faster memories such as caches are placed close to the processor.

2. Cost and Capacity

*There is a trade-off between **speed, capacity, and cost**:*

- *Registers → fastest, smallest, expensive*
- *L1 cache → very fast, small*
- *L2 cache → larger but slower*
- *L3 cache → larger and slower*
- *Main memory → much larger but slower*
- *Secondary storage → very large but much slower*

Thus, frequently used data is kept in smaller and faster memory.

3. Principle of Locality

Programs normally exhibit two important types of locality:

- ***Temporal locality:** Recently used data is likely to be used again.*
- ***Spatial locality:** Data located near recently accessed data is likely to be accessed soon.*

Caches exploit these properties to reduce the average memory access time.

Memory Hierarchy

Faster

↑

Registers

|

L1 Cache

|

L2 Cache

|

L3 Cache

|

Main Memory

|

Secondary Storage

↓

Larger

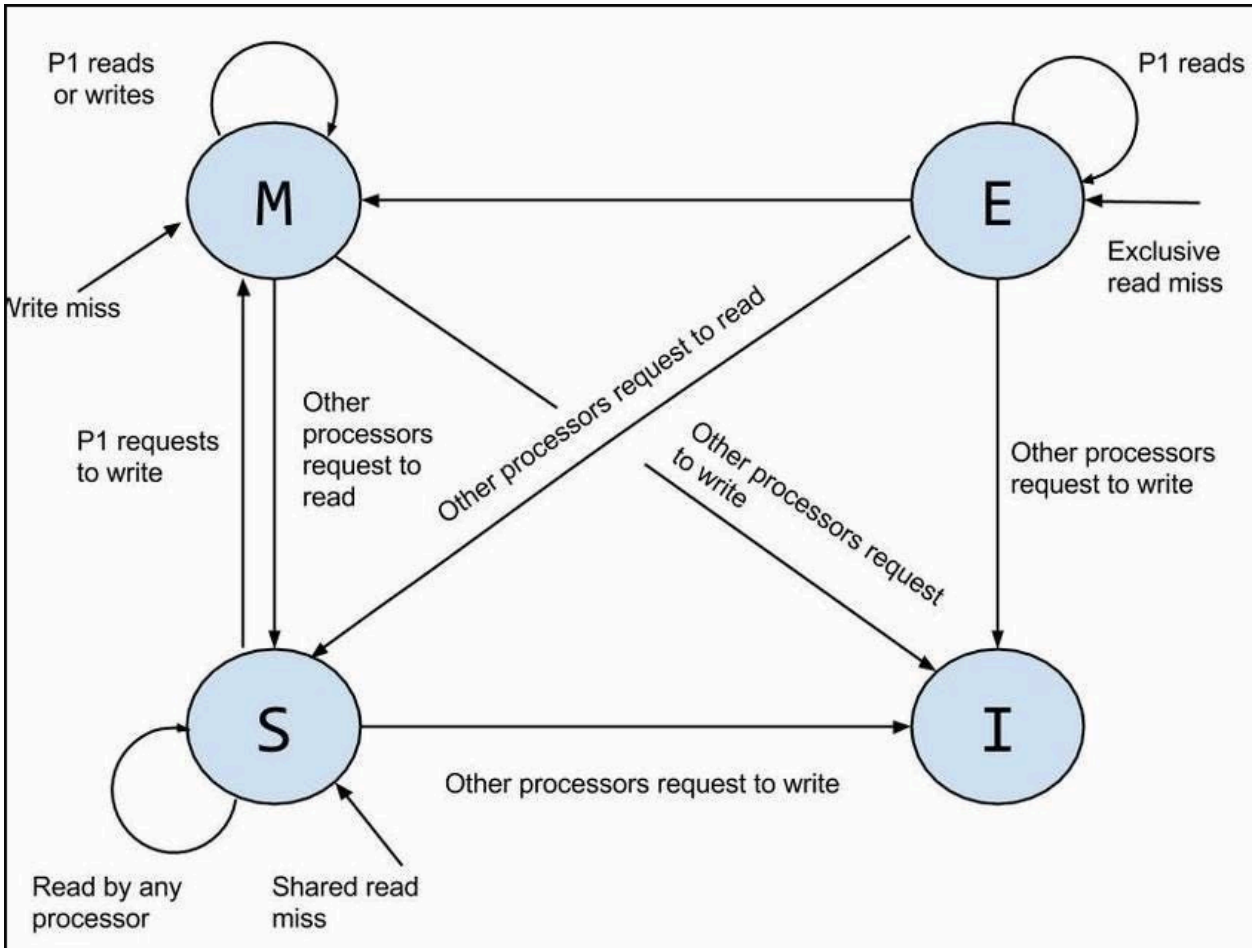
---

3. Multicore Architectures Based on Cache Organization

*Multicore processors can be classified according to whether their caches are **private, shared, or a combination of both**.*

A. Private Cache Architecture

*In a **private cache architecture**, each core has its own dedicated cache.*



Advantages

- *Very fast access to local cache.*
- *Cores do not directly compete for private cache space.*
- *Good performance for independent tasks.*

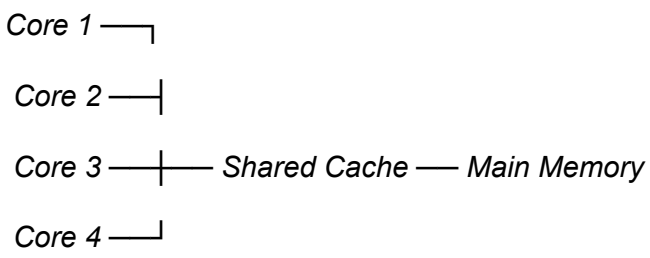
Disadvantages

- *The same data may exist in multiple caches.*
- ***Cache coherence** becomes necessary.*
- *Hardware protocols such as **MESI/MOESI** may be used to keep cached copies consistent.*

***Example:** If Core 1 changes a shared variable, the system must ensure that Core 2 does not continue using an outdated copy.*

B. Shared Cache Architecture

*In a **shared cache architecture**, multiple cores use the same cache, commonly an L2 or L3 cache.*



Advantages

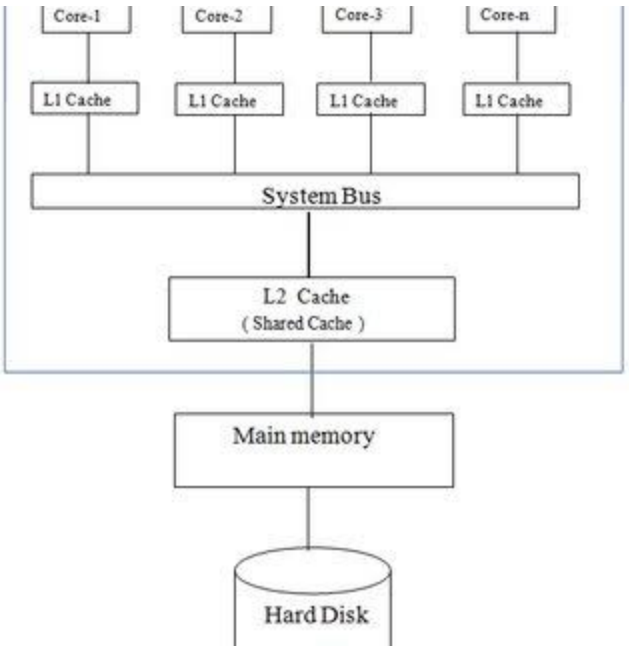
- *Cache space can be dynamically shared among cores.*
- *Reduces duplication of shared data.*
- *Data can be easily shared between cores through the common cache.*

Disadvantages

- *Multiple cores may compete for cache capacity and bandwidth.*
- *Heavy workloads can create cache contention.*
- *Shared-cache access may be slower than a private L1 cache.*

C. Hybrid Cache Architecture

*A **hybrid architecture** combines private and shared caches. This is common in modern multicore processors.*



Features

- ***L1:*** *Usually private to each core and very fast.*
- ***L2:*** *Often private to each core and larger than L1.*
- ***L3:*** *Usually shared by all cores and larger than L2.*
- ***Main memory:*** *Shared by all cores.*

Advantages

1. *Very fast access through private L1/L2 caches.*
2. *Shared L3 provides efficient communication and data sharing.*
3. *Reduces the need to access slower main memory.*
4. *Provides a good balance between performance, capacity, and cost.*

Disadvantage

The system requires ***cache-coherence mechanisms*** *to ensure that different cores have a consistent view of shared data.*

Comparison

Feature	***Private Cache***	***Shared Cache***	***Hybrid Cache***

Cache ownership	*Each core has its own cache*	*Cache shared by cores*	*Private + shared*
Access speed	*Very fast*	*Relatively slower*	*Fast*
Cache contention	*Low*	*Higher*	*Moderate*
Data duplication	*Possible*	*Reduced*	*Controlled*
Coherence complexity	*Higher*	*Lower at shared level*	*Requires coherence*
Flexibility	*Lower*	*High*	*High*
Common use	*Individual core caches*	*Shared L3*	*Modern multicore processors*

Conclusion

*A **multicore system** contains multiple processing cores on a single chip and uses a **hierarchical memory system** to reduce the large speed gap between processors and main memory.*

The three major cache organizations are:

> ***Private cache:** Each core has its own cache.*
>
> ***Shared cache:** Multiple cores use a common cache.*
>
> ***Hybrid cache:** Each core has private lower-level caches, while a higher-level cache is shared.*

Hybrid architecture is widely used because it combines the low latency of private caches with the efficient data sharing of a shared cache.

- Briefly describe the different cache coherence protocols and their role in maintaining data consistency. *(CA / Old)*

- Briefly explain the key design considerations for cache memory and management. *(CA / Old)*

5.6 GPU Architectures and GPGPU

- What is a GPU? Explain the basic architecture of GPU. *(2025 Fall Mock & 2025 Fall New)*

What is a GPU?

*A **Graphics Processing Unit (GPU)** is a specialized processor designed for **highly parallel computation**. Originally developed for graphics rendering, modern GPUs are also used in **GPGPU computing, HPC, machine learning, and data-intensive applications**. Unlike CPUs, which use a few powerful cores, GPUs use **thousands of smaller cores** to achieve high **throughput** and hide memory latency.*

Basic Architecture of GPU

The basic architecture consists of the following components:

1. ***Streaming Multiprocessors (SMs):***
   *The main processing units of a GPU. Each SM contains many **CUDA cores** that execute multiple threads in parallel.*
2. ***Warp Scheduler:***
   *Threads are organized into **warps**, normally containing **32 threads**. The warp scheduler manages and executes these threads efficiently.*
3. ***Shared Memory:***
   *A fast, low-latency **on-chip memory** within each SM. It allows threads in the same thread block to quickly share and reuse data.*
4. ***Global Memory:***
   *Large **off-chip DRAM** shared by all SMs. It provides high memory bandwidth but has higher latency than shared memory.*
5. ***SIMD/SIMT Execution:***
   *GPUs use **SIMD/SIMT-style parallel execution**, allowing many threads to perform similar operations on different data simultaneously.*

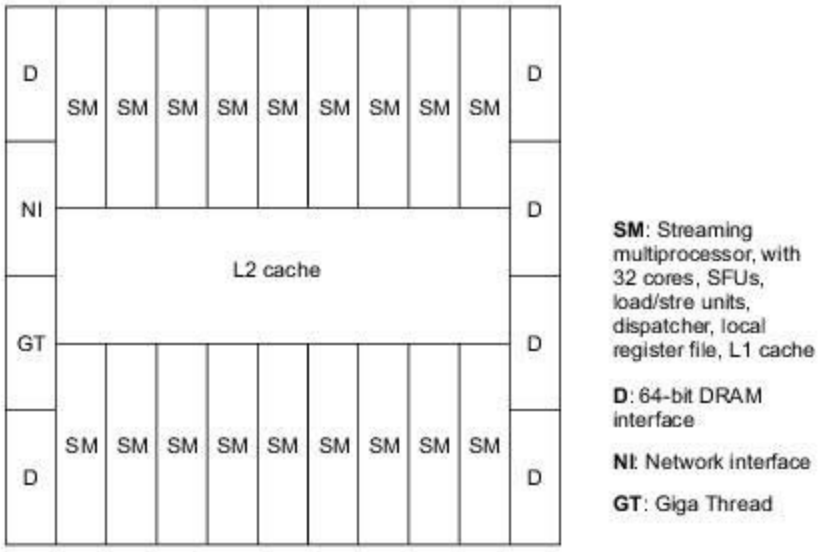


Fig. 13.7 Block diagram of Fermi GPU

In summary: *A GPU combines* ***SMs, CUDA cores, warp scheduling, shared memory, and high-bandwidth global memory*** *to execute thousands of threads concurrently and provide high computational throughput.*

Chapter 6: Multiprocessor Operating System

6.1–6.3 Multiprocessor OS & Scheduling

- What is a multiprocessor operating system? Discuss briefly on the classification of multiprocessor operating systems. *(2024 Fall)*

Multiprocessor Operating System

A ***multiprocessor operating system (OS)*** *is an operating system designed to manage and coordinate* ***two or more CPUs/processors*** *within a computer system. The processors may share* ***main memory and I/O devices*** *and can execute multiple processes simultaneously. This improves* ***processing speed, throughput, resource utilization, and reliability****.*

Classification of Multiprocessor Operating Systems

1. Based on Coupling

- ***Tightly Coupled Systems:*** *Multiple processors share common memory and are managed by a* ***single operating system****. Examples include* ***SMP and AMP****.*
- ***Loosely Coupled Systems:*** *Processors have their* ***own memory and OS*** *and communicate through* ***message passing****. Examples include distributed systems and clusters.*

2. Based on Processor Control

- ***Symmetric Multiprocessing (SMP):*** *All CPUs have* ***equal rights****. Any CPU can execute OS or user processes and access shared resources.*
- ***Asymmetric Multiprocessing (AMP):*** *CPUs have* ***different roles****. A master CPU controls system operations while slave CPUs execute assigned application tasks.*

3. Based on Supervisor Organization

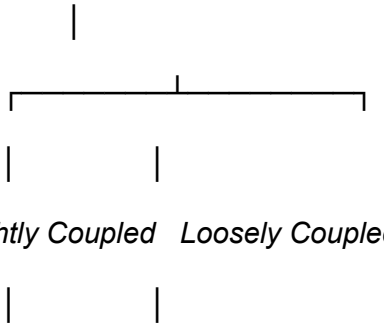
- ***Master-Slave OS:*** *One* ***master processor*** *controls and assigns tasks to slave processors.*
- ***Separate Supervisor System:*** *A dedicated processor performs* ***system supervision****, while other processors execute user applications.*
- ***Floating Supervisor System:*** *The* ***supervisor role can move dynamically*** *between processors, improving fault tolerance and load balancing.*

4. Specialized Multiprocessor Systems

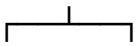
- ***Distributed Operating System:*** *Manages processors located on different machines and provides a* ***single-system view*** *to users.*
- ***Parallel Operating System:*** *Supports parallel execution on tightly coupled processors and is suitable for* ***scientific and engineering computations****.*
- ***Network of Workstations (NOW):*** *Connects independent computers through a network to share computational resources, providing a* ***cost-effective alternative to supercomputers****.*

Summary

Multiprocessor Operating System



Tightly Coupled Loosely Coupled



Distributed/

SMP AMP Cluster Systems

In short: *Multiprocessor OSs are mainly classified according to* ***coupling, processor control, supervisor organization, and system architecture****, with the common goal of improving* ***parallelism, throughput, resource utilization, and reliability****.*

- What is a multiprocessor operating system? Explain the software requirements of multiprocessor operating systems. *(2025 Fall Mock & 2025 Fall New)*

Multiprocessor Operating System

A ***multiprocessor operating system*** *is an OS designed to manage and coordinate* ***multiple processors (CPUs)*** *in a single computer system. The processors typically share* ***main memory, system buses, and I/O devices****.*

The OS assigns processes to different CPUs, allowing ***parallel execution****, which improves* ***throughput, resource utilization, performance, and reliability****.*

Software Requirements of Multiprocessor Operating Systems

1. Process Synchronization

Multiple processors may access shared data simultaneously. The OS must prevent ***race conditions and deadlocks*** *using:*

- ***Mutexes*** *for mutual exclusion.*
- ***Semaphores*** *for controlling and coordinating access to shared resources.*

2. Process Management and Scheduling

The OS must efficiently distribute processes among CPUs.

- ***Load balancing*** *distributes work evenly.*
- ***Gang scheduling*** *coordinates related processes or threads.*
- ***Thread support*** *allows multiple threads to execute in parallel.*

3. Memory Management and Consistency

*The OS must manage shared memory and maintain **memory consistency**. **Cache coherence** ensures that different processors do not use outdated copies of shared data.*

4. Inter-Processor Communication (IPC)

Processors and processes need mechanisms for exchanging information.

- ***Shared memory** provides fast communication through common memory.*
- ***Message passing** allows processes to exchange data explicitly.*

5. Fault Tolerance and Security

*The OS should detect and handle **processor failures** without bringing down the entire system. It must also provide **access control and protection** for memory, processes, and system resources.*

6. Support for Parallelism

The OS should support:

- ***Implicit parallelism:** The system/compiler automatically manages parallel execution.*
- ***Explicit parallelism:** Programmers explicitly define parallel tasks using technologies such as **OpenMP or MPI**.*

7. Scalability

*The OS should efficiently support an increasing number of processors and workloads without excessive **scheduling, synchronization, or communication overhead**.*

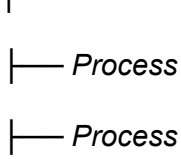
8. Virtualization Support

*Modern multiprocessor systems may support **virtual machines** through a **hypervisor/VMM**.*

- ***Type 1:** Runs directly on physical hardware.*
- ***Type 2:** Runs on top of a host operating system.*

Summary

Software Requirements



├── *Memory Management & Cache Coherence*

├── *Inter-Processor Communication*

├── *Fault Tolerance & Security*

├── *Parallelism Support*

├── *Scalability*

└── *Virtualization Support*

In conclusion: *A multiprocessor OS must efficiently manage* ***multiple CPUs, shared resources, synchronization, communication, memory consistency, parallel execution, and failures*** *to achieve high performance and reliable operation.*

- Compare and contrast Symmetric Multiprocessing (SMP) and Asymmetric Multiprocessing (AMP) in terms of process scheduling, resource sharing, and fault tolerance. Provide examples of real-world implementations. *(CA / Old)*

6.4 Distributed Scheduler - PVM & Loosely Coupled Systems

- Describe the working principle of PVM (Parallel Virtual Machine) as a distributed scheduler. How does PVM manage task allocation and communication across heterogeneous computing nodes? Illustrate with a practical scenario. *(2024 Fall)*

PVM as a Distributed Scheduler

Parallel Virtual Machine (PVM) *is a middleware platform that combines* ***heterogeneous computing nodes*** *and presents them as a single* ***virtual machine****. It divides a parallel application into independent* ***tasks*** *and manages their execution across different processors.*

Working Principle

1. ***Virtual Machine Abstraction:*** *PVM hides differences in hardware, operating systems, and processor architectures and provides a unified computing environment.*
2. ***Task-Based Execution:*** *Applications are divided into parallel* ***tasks****, and each task receives a unique* ***Task ID*** *for identification and communication.*
3. ***Task Allocation:*** *PVM supports* ***static and dynamic scheduling****. Computationally intensive tasks can be assigned to powerful nodes, while lighter tasks are assigned to slower nodes.*
4. ***Heterogeneous Nodes:*** *Different workstations, SMP systems, clusters, and processors can participate in the same virtual machine.*

5. ***Message Passing:*** *Since nodes have separate memory, tasks communicate using explicit* ***message-passing primitives*** *such as send and receive.*
6. ***Fault Tolerance:*** *PVM detects node failures and can reallocate interrupted work to available nodes.*

Practical Scenario: Seismic Data Processing

Suppose a research institute processes a large ***3D seismic dataset*** *using four heterogeneous nodes:*

- ***Node A:*** *Manager task*
- ***Node B:*** *Powerful SMP server*
- ***Node C:*** *Workstation cluster*
- ***Node D:*** *Older single-processor system*

The manager divides the dataset using ***domain decomposition****. PVM assigns larger computational tasks to Nodes B and C and smaller tasks to Node D. Each node processes its data in* ***private local memory*** *and sends results to Node A using* ***message passing*** *and Task IDs. If Node D fails, its task can be reassigned to an available node.*

Thus, PVM provides a unified virtual machine, dynamic task allocation, communication through message passing, and support for heterogeneous distributed computing.

- Loosely Coupled Systems *(Short Note, 2024 Fall)*
- What is a parallel virtual machine (PVM)? Explain its function. *(2025 Fall Mock & 2025 Fall New)*

Parallel Virtual Machine (PVM)

PVM *is a software* ***middleware framework and platform*** *designed for* ***distributed parallel computing****. It abstracts physically separate,* ***heterogeneous computer systems*** *and presents them as a* ***single, unified logical computer*** *or* ***virtual supercomputer****.*

Functions of PVM

- ***Resource Abstraction*** *– Aggregates diverse physical systems into a single virtual processing environment.*
- ***Distributed Task Scheduling*** *– Manages execution of* ***tasks*** *using static or dynamic scheduling.*
- ***Explicit Message Passing*** *– Provides message-passing primitives for* ***communication and synchronization*** *using unique* ***Task IDs****.*
- ***Dynamic Host Pool Configuration*** *– Allows physical machines to be dynamically* ***added or removed****.*

- ***Fault Tolerance & Resilience*** *– Provides* ***failure detection*** *and* ***process migration****.*
- ***Platform Independence*** *– Supports* ***C, C++, and Fortran*** *and provides portability across different hardware architectures.*

- Parallel Virtual Machine *(Short Note, CA / Old)*
- Distributed Scheduler *(Short Note, 2025 Fall New)*

6.5 PThreads & Thread Synchronization

- Describe the principles of thread synchronization in multithreaded systems. What are mutexes, semaphores, and barriers? *(2024 Fall)*

Principles of Thread Synchronization

Thread synchronization *is a coordination process that ensures multiple concurrent threads work together to reach a known point before continuing.*

- ***Critical Section (CS):*** *Segment of code accessing shared variables; executed by* ***one thread at a time****.*
- ***Mutual Exclusion:*** *At most* ***one thread*** *executes a critical section at any moment.*
- ***Race Conditions:*** *Multiple threads access and modify shared data simultaneously, causing unpredictable output.*
- ***Wait Protocols:*** *Blocked threads use:*
    - ***Busy Wait (Spin Lock):*** *Continually retries the lock; consumes CPU cycles.*
    - ***Sleep Wait (Suspend Lock):*** *Removed from active execution queue and placed in a wait queue.*

Mutexes

Mutex *is a software synchronization primitive designed to enforce* ***mutual exclusion****.*

- ***Acquire (lock):*** *Takes ownership; if already held, thread is blocked.*
- ***Release (unlock):*** *Relinquishes ownership.*
- ***Lock Ownership:*** *At most* ***one thread*** *can hold a mutex.*
- *Uses atomic hardware instructions such as* ***Test&Set, Swap, Load-Linked/Store-Conditional****.*

Semaphores

Semaphore *is a low-level signaling mechanism used to* ***coordinate thread execution*** *and control access to shared resources.*

- *Holds a* ***non-negative integer value****.*

- ***P (Wait):*** *If `s > 0`, decrements and proceeds; if `s = 0`, thread is suspended.*
- ***V (Signal):*** *Increments value and awakens a blocked thread.*
- ***Binary Semaphore:*** *`0` or `1`; restricts access to a single critical section.*
- ***Counting Semaphore:*** *Non-negative integer up to `n`; allows up to `n` threads to access shared resources.*

Barriers

Barrier *is a collective synchronization method that forces threads to wait until* ***every participating thread has reached a designated point****.*

- ***Waiting Phase:*** *Thread reaches barrier, updates barrier counter, and blocks.*
- ***Release Phase:*** *When the* ***last thread*** *reaches the barrier, all blocked threads are released.*
- *Used to coordinate* ***sequential stages*** *of parallel algorithms.*

- Explain the role of POSIX threads (PThreads) in shared memory systems. How does PThread synchronization (mutexes, condition variables) ensure safe and efficient parallel execution in a multiprocessor environment? *(CA / Old)*

POSIX Threads (Pthreads)

Pthreads *is a standardized* ***API for multi-threaded programming*** *on Unix-like operating systems. In shared memory systems, Pthreads provides software abstractions to exploit* ***hardware-level parallelism*** *on multiprocessor and multicore computers.*

Role of Pthreads in Shared Memory Systems

- ***Shared Address Space:*** *Threads share the* ***same virtual address space****, global variables, static data, and open file descriptors.*
- ***Implicit Communication:*** *Threads directly read/write shared memory without explicit* ***message passing****.*
- ***Low Overhead:*** *Thread creation and* ***context switching*** *are faster than processes.*
- ***Multiprocessor Parallelism:*** *Threads can execute on separate* ***CPU cores*** *using one-to-one thread mapping.*

Pthread Synchronization

1. Mutexes

Mutex (`pthread_mutex_t`) *is a synchronization lock used to enforce* ***mutual exclusion****.*

- *Protects **critical sections**.*
- *Allows **at most one thread** to access shared data at a time.*
- *Prevents **race conditions** and data corruption.*
- *Uses **sleep-wait**, freeing the CPU while a thread is blocked.*

2. Condition Variables

***Condition variables (`pthread_cond_t`)** provide **wait and signal communication** between threads.*

- *Thread waits until a specific **condition** becomes true.*
- *`wait` **atomically releases the associated mutex**.*
- *`signal` wakes one waiting thread.*
- *`broadcast` wakes all waiting threads.*
- *Eliminates **polling overhead**.*

In short

Pthreads → Shared address space → Implicit communication → Low overhead → Parallel execution

Mutexes → Mutual exclusion → Prevent race conditions

Condition variables → Wait/Signal → Coordinate threads efficiently